API Design and Data Modelling: a ride hailing service

Sonia Pepple
13 October 2026

Contents

Contents 2
About this document 4
Chapter 1. Requirements 5
1. Users 5
2. Five key actions 5
3. Rules that must always hold 5
4. Non goals 5
5. Open questions 5
Chapter 2. Entities and relationships 7
The relationships 7
The two richest entities 8
All sixteen entities 9
Conventions 10
Chapter 3. The seven hard questions 11
Question 1. Normalisation: where does each fact live? 11
Question 2. Money 11
Question 3. Status and state 12
Question 4. Time, deletion and personal data 12
Question 5. Identifiers 13
Question 6. Constraints 14
Question 7. Indexes 14
Summary. The seven answers 14
Chapter 4. The state machine 15
Allowed trip transitions 15
Allowed payment transitions 16
Forbidden trip moves 18
Forbidden payment moves 19
The rule for a stuck payment (move P3) 19
The way out of a stuck pending payment 19
Test of the trigger idea 20
Scheduled jobs 20
Chapter 5. Constraints and indexes 21
Invalid states that are now impossible 21
The catalogue check 23
Per entity constraints: payment as the model 23
Indexes 24
The statements that enforce the rules 24
An index added in Stage 4: the payments of a trip 24
Chapter 6. API contracts 26
Conventions 26
Format 26
Who calls 26
Ownership checks 26
Session settings the API sets for the database 26
The error body, and what never leaves the server 26
How a database error becomes an HTTP status 27
Idempotency in this API 27
A worked endpoint 27
The list contract 28
The endpoint matrix 29
The error catalogue 31
Chapter 7. Over fetching 32
The endpoint and why 32

The real response and its size 32
What each client uses 32
The GraphQL design sketch 33
When REST is still fine 34
Decisions 34
Chapter 8. Real time 35
The place and the choice 35
Comparing Server Sent Events and WebSockets 35
The recommendation 36
The feed and the LISTEN and NOTIFY tests 36
The event format, resume and ending 37
Chapter 9. The proof: plans and the three rejected inserts 38
The two heaviest queries 38
The three rejected inserts 39
The limits 40
Chapter 10. Open items and decisions 41
Decisions recorded 41
Approved 41
Still open 41
Placeholders 41
Known limits 42
Appendix 43
Appendix A, documents 43
Appendix B, migrations 43
Appendix C, evidence 43
Appendix D, queries and seeds 44
Appendix E, full text of two endpoints 44
EP02 POST /v1/trips (rider app; A1, B0, B1, B6) 45
EP15 POST /v1/trips/{trip_id}/payments (rider app; A4, B3, B6, U1) 45
Appendix F, entity field tables 46
rider (source: 02_entities.md lines 21 to 34) 46
driver (source: 02_entities.md lines 36 to 49) 47
vehicle (source: 02_entities.md lines 51 to 66) 47
trip_status_history (source: 02_entities.md lines 95 to 112) 47
location_ping (source: 02_entities.md lines 118 to 130) 48
fare_quote (source: 02_entities.md lines 132 to 149) 48
refund (source: 02_entities.md lines 168 to 181) 48
rating (source: 02_entities.md lines 183 to 194) 48
idempotency_key (source: 02_entities.md lines 196 to 212) 49
setting (source: 02_entities.md lines 214 to 225) 49
support_admin (source: 02_entities.md lines 229 to 240) 49
audit_log (source: 02_entities.md lines 242 to 258) 49
status_transition_rule (source: 02_entities.md lines 264 to 274) 50
provider_event (source: 02_entities.md lines 278 to 294) 50

About this document

Built from docs/05_traceability.md (its first paragraph), docs/10_brief_checklist.md (section 4, the summary) and docs/09_open_items.md (section 6, owner decisions recorded).

This document describes what was built: a ride hailing database of 16 tables with 116 constraints, 42 indexes, 27 triggers and 18 functions, CONFIRMED in evidence/catalogue_check_016.txt, together with the API contracts (27 endpoints EP01 to EP27 and 3 internal jobs JB1 to JB3, in docs/06_api_contracts.md). No API was built, so nothing was run over HTTP. The brief checklist summary records requirements 1, 2, 3, 6, 7 and 8 as DONE, requirement 4 as PARTIAL (D5, D13 and D16 have written design assumptions but are still open owner decisions) and requirement 5 as PARTIAL (four class notes unfilled, and the GraphQL part not run). What closes the gaps is that the owner supplies six required images (three diagrams and three screenshots of the rejected inserts) and two optional plan screenshots, fills four class notes, and confirms or replaces the written assumptions for D5, D13 and D16.

To follow the document, read the labels first. CONFIRMED means the output was run and kept in evidence/; INFERRED means the claim was reasoned, not run. The requirement IDs let a reader follow a rule from requirement to table to test: U1 to U4 are the four users, A1 to A5 the five key actions, B0 to B10 the rules, and Q1 to Q6 the questions, and docs/05_traceability.md maps every requirement ID against what serves it.

Chapter 1. Requirements

docs/01_requirements.md

Riders request trips and see a fare estimate. Drivers accept and drive them; riders pay through an outside provider. Both sides rate each other; support can investigate any trip.

1. Users

- U1 Rider.
- U2 Driver.
- U3 Support admin: reads trips and payments, cancels, refunds.
- U4 Payment provider: external system; may report results twice or late.

2. Five key actions

Proposal kept unchanged: the five cover the money path. Cancellation (B9) is a state change, not a sixth action; the state machine handles it (B5).

- A1 Rider requests a trip (pickup, dropoff, estimate). Success: one "requested" trip with a stored estimate. Fails: missing locations, outside service area, rider already active (B1), retry (B6).
- A2 Driver accepts. Success: exactly one driver assigned. Fails: two accept at once, driver already active (B2), rider cancelled first.
- A3 Driver moves trip to arrived, started, completed; rider watches. Success: steps recorded in order. Fails: steps skipped or repeated, location updates lost.
- A4 Rider pays. One payment covers exactly one fare or one cancellation fee. Fails: declined card, timeout, duplicate callback. A retried payment request creates no second charge (B6).
- A5 Both rate each other. Success: one rating per person per trip. Fails: trip not completed, window passed, rated twice (B7).

3. Rules that must always hold

Trip statuses: requested, assigned, arrived, started, completed, cancelled. Completed and cancelled are terminal. Refunds are rows on the payment side. A dispute is only a support note in the audit log.

- B0 A trip is ACTIVE in the statuses requested, assigned, arrived and started.
- B1 A rider has at most one active trip. Enforced by a database unique index.
- B2 A driver has at most one active trip. Same enforcement. A trip with no driver yet counts for the rider only.
- B3 Money is whole kobo in integer columns, currency NGN only. Enforced by column types and a currency check. A decimal amount is stopped by API validation, not by the column type (the database rounds 10.5 kobo to 11; see docs\04 question 2).
- B4 After completion, fare, distance, start and end times, rider and driver are frozen. Only updated_at may change. Enforced by a database trigger. Corrections and partial refunds are new rows.
- B5 Every status change is recorded (who, when, from, to); only legal changes are allowed.
- B6 Idempotency keys are unique per rider and action, not global. A retried request must not create a second trip or a second charge; it returns the stored result unchanged. Provider payment references are unique.
- B7 A rating needs a completed trip and a participant, within 7 days of completion, once per person per trip.
- B8 Personal data (names, phone numbers, locations) follows Nigeria's NDPA 2023: collect only what is needed, allow deletion, restrict access. Retention periods are NOT verified and need legal advice.
- B9 Rider, driver or support may cancel from requested, assigned or arrived. The system cancels a requested trip no one accepted after request_timeout_seconds, with no fee (placeholder 600 seconds, needs owner approval, held in a settings row). After started, only support. A flat fee applies after assignment, none before. The fee is 50000 kobo (500 naira), a placeholder approved by the owner and held in a settings row, not hard coded.
- B10 The sum of refunds on a payment never exceeds the payment amount.

4. Non goals

- Matching algorithm, a live map or route display, surge pricing, promo codes, driver payouts, tax, multi city and mobile apps are out of scope.
- The rider sees the driver's latest position through the event stream (EP27), but no map or route is drawn.
- A full dispute system is out of scope, and a money record for a manual refund of a failed payment is a known gap: its trace is provider_event plus audit_log.

5. Open questions

ID	Question	Status
Q1	Currency: NGN only, kobo (B3).	ANSWERED
Q2	Cancellation fee: flat after assignment (B9).	ANSWERED

Q3	Rating window: 7 days (B7).	ANSWERED
Q4	Refunds: partial allowed, as new rows (B4).	ANSWERED
Q5	Location retention: blocked on legal advice. No retention period is chosen.	OPEN
Q6	An assigned driver never arrives and the rider is charged the fee if they cancel. Needs an owner decision before launch. Recommended: no fee when the driver is more than a stated number of minutes late, held in a setting. Not built here.	OPEN

Q5 (location retention) and Q6 (a driver who never arrives) are left open on purpose; they are carried forward to chapter 10.

Chapter 2. Entities and relationships

Built from docs/02_entities.md and docs/02b_relationships.md.

The relationships

Field names are exactly those in docs/02_entities.md. "FK" means foreign key. The FK sits on the "many" side and points at the "one" side. Every FK uses ON DELETE RESTRICT, because rows are never hard deleted when they hold personal or financial history, so a delete that would orphan a row should fail loudly. The table below lists every related pair, R1 to R20.

id	relationship	cardinality
R1	rider to fare_quote	one to many
R2	rider to trip	one to many
R3	rider to idempotency_key	one to many
R4	driver to vehicle	one to many
R5	driver to trip	one to many (optional: a trip has 0 or 1 driver)
R6	vehicle to trip	one to many (optional)
R7	fare_quote to trip	one to zero or one
R8	trip to trip_status_history	one to many
R9	trip to location_ping	one to many
R10	trip to payment	one to many (at most one live fare payment and one live fee payment)
R11	payment to refund	one to many
R12	trip to rating	one to many (at most two: one per role)
R13	support_admin to refund	one to many
R14	support_admin to setting	one to many (optional)
R15	rider / driver / support_admin to trip_status_history	one to many each (exactly one of the three is set, or none for system)
R16	rider / driver / support_admin to audit_log	one to many each (exactly one set, or none for provider and system)
R17	trip to idempotency_key	one to many (optional)
R18	payment to idempotency_key	one to many (optional)
R19	rider to trip, via fare_quote	the quote must belong to the trip's rider
R20	payment or refund to provider_event	one to many, by value only

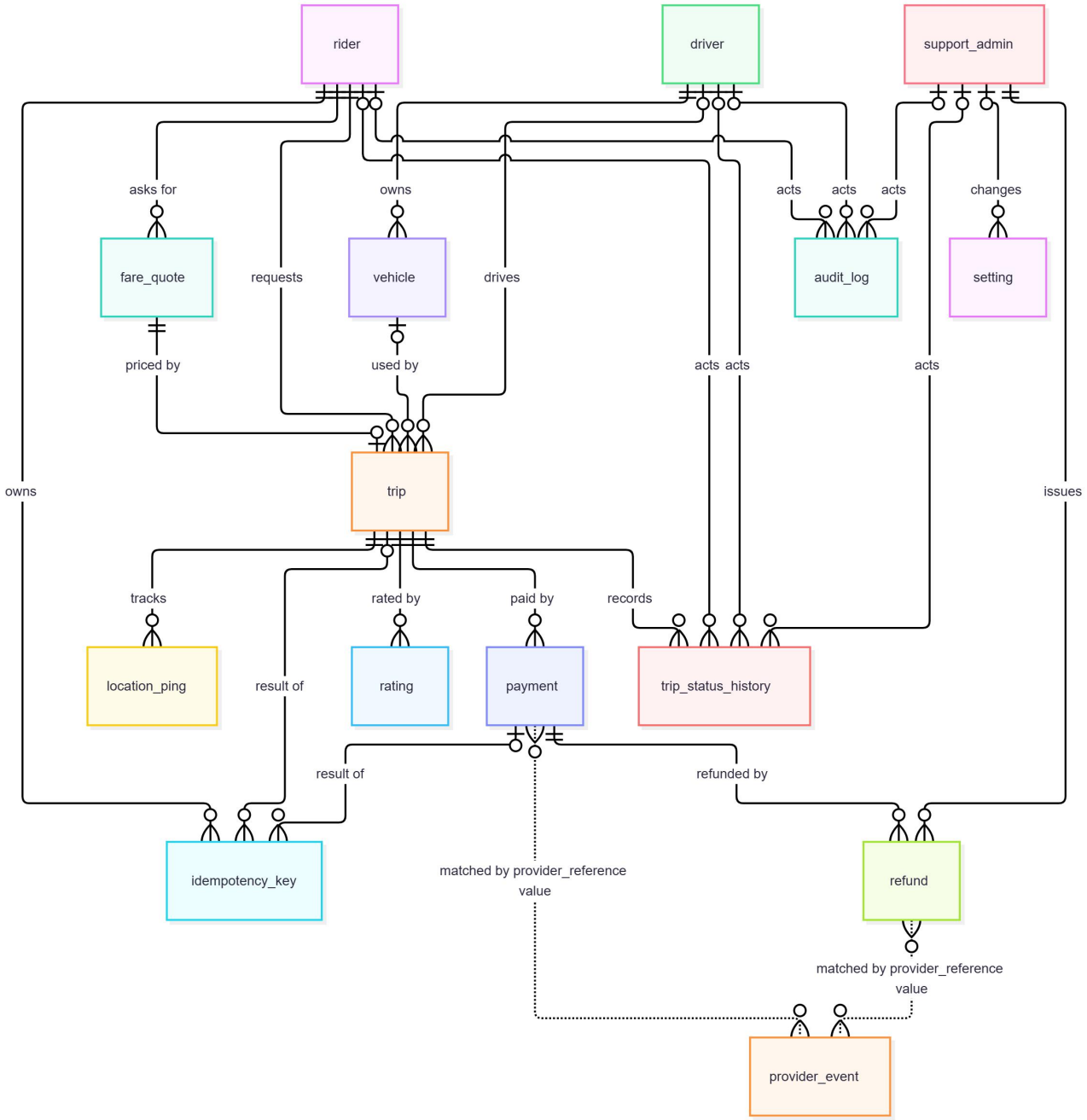


Image 2.1. The entity relationship diagram

The two richest entities

The trip and the payment tables carry the most fields, so they are shown in full below. A trip is one ride from request to a final state, and it is also the join between a rider and a driver. A payment is one attempt to collect one fare or one cancellation fee through the payment provider (A4).

Field	Type	Notes
id	uuid	primary key
rider_id	uuid	foreign key to rider; never changes
driver_id	uuid	foreign key to driver; empty until assigned (B2); never changes once set
vehicle_id	uuid	foreign key to vehicle; empty until assigned; must belong to the same driver
fare_quote_id	uuid	foreign key to fare_quote; one quote makes at most one trip
status	text	requested, assigned, arrived, started, completed, cancelled (default requested)
pickup_address	text	the address text the rider confirmed; the only copy
dropoff_address	text	same
driver_name_snapshot	text	HISTORY snapshot of driver.full_name at assignment
vehicle_plate_snapshot	text	HISTORY snapshot of vehicle.plate_number at

		assignment
started_at	timestamptz	set when status becomes started
completed_at	timestamptz	set when status becomes completed
cancelled_at	timestamptz	set when status becomes cancelled
cancelled_by_role	text	rider, driver, support or system (the timeout in B9); set when cancelled
cancellation_fee_minor	bigint	kobo; set when cancelled (0 means no fee); copied from the setting at that moment
cancellation_fee_currency	char(3)	NGN; set with the fee
fare_amount_minor	bigint	final fare in kobo; set when completed
fare_currency	char(3)	NGN; set with the fare
distance_m	integer	driven distance in metres; set when completed
created_at	timestamptz	this is the moment of request
updated_at	timestamptz	the only field that may change after completion (B4)

Field	**Type**	**Notes**
id	uuid	primary key
trip_id	uuid	foreign key to trip; the rider is found through the trip
purpose	text	fare or cancellation_fee
amount_minor	bigint	kobo; more than 0; must equal the trip's fare or fee
currency	char(3)	NGN
status	text	pending, succeeded, failed (default pending). The provider moves it out of pending. Support may only fail a pending payment after a payment_not_found provider_event exists for it (03, move P3)
provider_reference	text	the provider's own id for this payment; unique (A4). Matches provider_event.provider_reference by value, not by foreign key
paid_at	timestamptz	required when succeeded
failure_reason	text	required when failed
created_at	timestamptz	
updated_at	timestamptz	

All sixteen entities

The entity list I was given has 14 entries. I added two and dropped none, so the total is 16 entities. The two added are status_transition_rule (entity 15), which keeps the legal status changes (B5, B9) as rows a person can read, query and review, and provider_event (entity 16), which stores one row per message from the payment provider so a repeated or contradicting message (U4, B6) is recognised.

Entity	**Identifier**	**Number of fields**	**What it is for**
rider	id	9	A person who requests and pays for trips.
driver	id	9	A person who accepts and drives trips.
vehicle	id	11	A car that belongs to one driver.
trip	id	21	One ride from request to a final state; also the join between a rider and a driver.
trip_status_history	id	11	One row for every status change of a trip, written by the database itself (B5). Append only.
location_ping	id	8	One GPS point sent by the driver's phone while a trip is in progress. Append only except for scheduled deletion.
fare_quote	id	13	The price estimate shown to the rider before the trip exists. It is never edited.
payment	id	11	One attempt to collect one fare or one cancellation fee through the payment provider (A4).
refund	id	9	Money returned to the rider against one payment, whole or part. Never edited.
rating	id	7	One person's rating of the other after a completed trip. Never edited.

idempotency_key	id	12	A client chosen key that makes a retry safe. Stores the first answer so a retry gets it back unchanged.
setting	key (text)	7	Named whole number settings that the owner can change without a code release.
support_admin	id	7	A staff member who investigates trips, cancels and refunds.
audit_log	id	12	A permanent record of sensitive actions: setting changes, payment status changes, refunds, cancellations by support, and support notes about disputes. Append only.
status_transition_rule	the whole row (entity, from_status, to_status, actor_role)	6	One row per legal status change for a given actor, for the trip and for the payment.
provider_event	id (our uuid)	12	One row for each message the payment provider sends us (a payment or refund result).

The field tables of the other 14 entities are not reproduced here. They stay in docs/02_entities.md and are listed in the appendix by name.

Conventions

Every table follows the same conventions. Every table has an identifier, the column id of type uuid, which is a random uuid rather than a sequential number. Every table also has created_at and updated_at, both timestamptz NOT NULL DEFAULT now(), and a shared trigger set_updated_at refreshes updated_at on every update. On the append only tables no update is ever allowed, so updated_at always equals created_at, and it is kept only so every table looks the same. Money is a bigint count of kobo with a char(3) currency column beside it, so no float or numeric is used for money. Latitude is numeric(8,6) and longitude is numeric(9,6), chosen so the same value always reads back exactly. Required means NOT NULL and optional means NULL allowed. Two tables are the exceptions to the id rule: setting uses the text column key as its primary key, and status_transition_rule uses the whole row (entity, from_status, to_status, actor_role) as its primary key.

Three links are deliberately not foreign keys. The first is audit_log.entity_id, which points at different tables depending on entity_type, and a foreign key cannot do that, so integrity comes from the trigger that writes the row. The second is status_transition_rule, which is not linked to trip.status or payment.status because a foreign key cannot check a pair of values (from and to); instead a trigger reads the rule rows. The third is provider_event.provider_reference, which is not linked to payment or refund by a foreign key; this is the same by value link as R20, because the same column can point at either table and a message about a reference we do not know must still be stored so support can see it (U4, B6), with the match done by value in the callback handler.

Chapter 3. The seven hard questions

docs/04_hard_questions.md

Each answer has: question, answer, reason, requirement IDs, enforcer. CONFIRMED means I ran it and kept the output (the numbered tests are listed in the report). INFERRED means I reasoned it. Table and field names match 02_entities.md. The status rules are in 03_state_machine.md.

Question 1. Normalisation: where does each fact live?

Each fact lives in exactly one home table, and everything else finds it by a foreign key. Exceptions are the deliberate denormalisations below, each with its own reason and its own way of staying correct.

Fact	Home	How others find it
A person's name, phone	rider, driver, support_admin	trip.rider_id, trip.driver_id, actor columns
Who paid for a trip	trip.rider_id	payment has no rider column; join through trip
Who is rated, and who rated	trip (rider_id, driver_id) + rating.rater_role	rating has no ratee column; join through trip
Whose car it is	vehicle.driver_id	trip.vehicle_id, with the composite FK
Pickup and dropoff coordinates	fare_quote	trip joins to its quote; not copied
Pickup and dropoff address text	trip (the only copy)	n/a
Fare price rules, fee, rating window	setting	read at the time they are needed
Legal status moves	status_transition_rule	read by triggers

No copy means no copy can disagree (B4, B6). The one copy rule also makes erasure under B8 possible: a name stored in six tables must be found and cleared in six tables.

```
Denormalisation 1: HISTORY snapshot.
trip.driver_name_snapshot and trip.vehicle_plate_snapshot copy the driver's
name and the car's plate at the moment of assignment. Also
trip.cancellation_fee_minor copies the setting at the moment of cancellation.
Why: these are facts about the past. A driver may later change name, or sell
the car, or the owner may change the fee. An old receipt must still say what
was true then (B4).
Denormalisation 2: SPEED cache.
rider.rating_sum, rider.rating_count, driver.rating_sum, driver.rating_count
hold running totals of the stars each person received.
Why: showing "4.8 stars" on every trip screen would otherwise mean adding up
thousands of rating rows each time. I store integer totals, not an average.
```

The two real denormalisations are the HISTORY snapshot and the SPEED cache. The derived copies trip.started_at, trip.completed_at and trip.cancelled_at repeat times that also exist in trip_status_history; the history row is the original, and the trip column can always be recomputed from it. They are written by the same update that makes the history row (triggers trip_enforce_transition and trip_write_history), so they agree.

The repair and detect query below recomputes the truth from the rating table. Run it daily and alarm if it returns any row. It has not been scheduled, because no job runner exists (the tables exist now; the query is INFERRED to be correct by reading it, not by running it on a schedule). See docs/10_brief_checklist.md section 3, the strictness note.

```
-- Drivers whose cache disagrees with the rating rows (rider side is the mirror image).
SELECT d.id, d.rating_sum, d.rating_count,
       COALESCE(SUM(r.stars),0) AS true_sum, COUNT(r.id) AS true_count
FROM driver d
LEFT JOIN trip t   ON t.driver_id = d.id
LEFT JOIN rating r ON r.trip_id = t.id AND r.rater_role = 'rider'
GROUP BY d.id
HAVING d.rating_sum <> COALESCE(SUM(r.stars),0) OR d.rating_count <> COUNT(r.id);
```

IDs. B4, B5, B7, B8, A5. Enforcers. triggers trip_enforce_transition, trip_write_history, rating_update_cache; checks trip_snapshots_match_assignment, rider_rating_cache_valid, driver_rating_cache_valid.

Question 2. Money

Money is a whole number of kobo in a bigint column, with a char(3) currency column beside it, always NGN.

Every money field in 02_entities.md:

Table	Amount field	Currency field
fare_quote	amount_minor	currency
trip	fare_amount_minor	fare_currency
trip	cancellation_fee_minor	cancellation_fee_currency
payment	amount_minor	currency
refund	amount_minor	currency
provider_event	amount_minor (optional)	currency (optional, filled together with the amount)
setting	int_value, only for the keys ending in _minor (cancellation_fee_minor,	currency

	base_fare_minor, per_km_minor, per_minute_minor)	

That is 7 amount columns, each with its own currency column beside it (7 currency columns). provider_event.amount_minor was added in this fix, so the count went from 6 to 7. The setting column holds four different money keys, but they share one column. fare_quote.estimated_distance_m and trip.distance_m are distances, not money.
The test is CONFIRMED, a rolled back transaction on ridehail_design, PostgreSQL 16.15: a value of 10.5 kobo becomes 11, and 11.5 becomes 12, with no error, while the text "10.5" is refused as an invalid bigint. What that means for the API, in order: API validation is the only layer that can reject 10.5, by demanding a whole number (JSON type integer) before the value reaches the database, and this is application code. The bigint type blocks decimals only for text typed parameters, which is a side effect, not a guarantee. A check constraint cannot help, because it runs after the rounding and only sees 11.
Reason. Binary floats cannot hold 0.1 exactly, so sums of floats drift by fractions of a kobo. Whole kobo add and compare exactly. Fares computed from per kilometre prices need one rounding rule; that rule belongs in one function in the API stage, not in a trigger (INFERRED; the contract now states one rounding rule, applied once at the end, in 06 EP01, design assumption D16).
IDs. B3, B10, A1, A4, Q1. Enforcers. API validation (decimals); check constraints (sign, currency); the column type (storage).

Question 3. Status and state

The full tables, forbidden moves, diagrams and the trigger test are in 03_state_machine.md. The rules from that table that matter most, and the enforcer for each:

Rule	Enforcer
One active trip per rider (B0, B1)	partial unique index trip_one_active_per_rider
One active trip per driver (B0, B2)	partial unique index trip_one_active_per_driver
Support fails a stuck payment only with proof (A4, B6)	rule row P3 (actor support) + trigger payment_enforce_rules, which looks up a payment_not_found row in provider_event (03 section 4b, tested)
A completed trip is frozen (B4)	trigger trip_enforce_transition; deletes blocked by trip_block_delete
Payment moves need a rule row (A4)	table status_transition_rule + trigger payment_enforce_rules
Refunds never exceed the payment (B10)	trigger refund_enforce_total

The remaining rules of this table, and their enforcers, are covered in chapter 4 and chapter 5.
The rating rule (a rating needs a completed trip) is NOT in this table; it is found in the table of invalid states in chapter 5.
Reason. See section 5 of 03_state_machine.md. In short: checks see one row, triggers see old and new rows and other tables, unique indexes are atomic.
IDs. B0, B1, B2, B4, B5, B6, B9, B10, A1 to A4. Enforcer. as listed.

Question 4. Time, deletion and personal data

Every table has created_at and updated_at (timestamptz, NOT NULL, default now()). The trigger set_updated_at refreshes updated_at on tables that are updated: rider, driver, vehicle, trip, payment, provider_event, idempotency_key, setting, support_admin. On the append only tables updated_at equals created_at for ever (decision 3). I use timestamptz so a stored time is an exact moment, not a clock reading without a zone.

Table	Delete style	Reason
rider, driver, vehicle, support_admin	Soft delete: set deleted_at, blank personal fields	Other rows point at them. A hard delete would break the foreign keys or destroy the financial record (B4). A soft delete lets the person leave (B8) while the money trail stays.
trip, payment, refund, provider_event, trip_status_history, fare_quote, rating, audit_log	Never deleted	These are the financial and audit record (B4, B5, B10). Personal text inside them is blanked, not the row.
location_ping	Hard delete once a retention period exists	Raw location is the most sensitive data and has no value after the trip is settled. No period is chosen (Q5), so no deletion job is designed yet.
idempotency_key	Hard delete after expires_at	It only exists to make a retry safe for a short time.
setting, status_transition_rule	Not deleted in normal use	Reference data. Changes to settings are

		written to audit_log by trigger setting_write_audit.

What happens to a deleted rider's trips and payments. They stay. The rider row is anonymised: full_name becomes the text Deleted rider, phone becomes the word deleted followed by the rider's id (so it is still unique and still not empty), email is cleared, deleted_at is set. The rider's trips, payments and refunds keep their ids and amounts, because a business must keep a record of money (B4). The rider's pickup_address and dropoff_address on old trips are replaced with the text erased, and rating comments are cleared. The same is done for a driver (driver_name_snapshot, vehicle_plate_snapshot; the driver's vehicles get deleted_at and a blanked plate). A person with an active trip cannot be erased until the trip ends. This is now checked inside the erasure function, so it is a database rule, not application code (tested for riders, step X6 below).
A conflict you must approve (decision 1). B4 as edited says that after completion only updated_at may change. Blanking an address on a completed trip changes more than updated_at. So B4 and B8 pull against each other. My recommendation: keep B4 for every money, distance, time, party and status field, and allow exactly five text columns to be blanked: pickup_address, dropoff_address, driver_name_snapshot, vehicle_plate_snapshot (on trip) and comment (on rating). Only a dedicated erasure function may do it. If you reject this, the alternative is to keep addresses for ever and rely on a lawful exemption, which I cannot judge.
How the erasure right is protected. The earlier app.erasure flag idea was weak, because any code that could talk to the database could set it, so the new design ties the right to a database role. Role ridehail_erasure_owner is NOLOGIN (nobody can connect as it) and not a superuser; it owns the functions erase_rider(uuid) and erase_driver(uuid), which are SECURITY DEFINER (they run with their owner's rights) with a fixed search_path so a caller cannot trick them into using another table. Role ridehail_app is the role the API connects as; it may EXECUTE the two functions and has no other route to the erased columns, and EXECUTE is revoked from everyone else (PUBLIC). The freeze trigger on trip (inside trip_enforce_transition) and the trigger rating_block_change allow those five columns to change only when current_user is ridehail_erasure_owner, and refuse any other column. The dev login ridehail used in this project is a superuser (CONFIRMED: rolsuper = t), and a superuser can bypass anything, so the API must never connect as ridehail. That is a deployment rule for Stage 3.
Gaps I did not solve. Pickup and dropoff coordinates inside fare_quote are also personal data and the quote is append only, so this design does not erase them. Retention periods for every personal field (and the NDPA's rules on erasure versus keeping financial records) are NOT verified and need legal advice (B8, Q5).
Soft delete against hard delete, and the compliance reasoning behind the choice. People are anonymised, not removed: rider, driver, vehicle and support_admin take a soft delete (deleted_at is set) and their personal fields are blanked, because other rows point at them and the money trail must stay (B4, B8). Financial and audit rows are kept for ever: trip, payment, refund, provider_event, trip_status_history, fare_quote, rating and audit_log are never deleted, because they are the financial and audit record (B4, B5, B10), and only the personal text inside them is blanked. Raw location is deleted after a retention period: location_ping is the most sensitive data and has no value once the trip is settled, so it is a hard delete, and because the period is not chosen yet no deletion job is designed. Short lived keys are deleted: idempotency_key is a hard delete after expires_at, because it only exists to make a retry safe for a short time. The legal position is not verified: this is a design view, not legal advice, the Nigeria Data Protection Act 2023 was not read for this task, the balance between a right to erasure and a duty to keep financial records is not settled here, and a qualified adviser must confirm the retention periods and the soft against hard delete choice before it is relied on.
IDs. B8, B4, B5, B10, Q5. Enforcers. trigger set_updated_at; trigger append_only_guard; trigger trip_block_delete; foreign keys with ON DELETE RESTRICT; functions erase_rider and erase_driver with role ridehail_erasure_owner; role ridehail_app (execute only).

Question 5. Identifiers

Every record id is a random UUID, generated by the database with gen_random_uuid(), not a counting number. Exceptions are setting.key and status_transition_rule's four column key, which are names, not records.
Security reason. A counting id (1, 2, 3...) lets a stranger try 4417, 4418 and read the neighbours' trips, and it tells competitors how many trips you have. A random id of 122 random bits cannot be guessed in practice. A random id is NOT authorisation: anyone who learns an id (from a log, a screenshot, a shared link or a browser history) can send it, so every endpoint must still check ownership. A rider may read a trip only if trip.rider_id is their own id, a driver only if trip.driver_id is theirs, and support only with a support role. Without that check the random id gives no protection at all. These checks are application code, and the database cannot do them for you.
Generation method and test (CONFIRMED, rolled back transaction, PostgreSQL 16.15): three inserted rows each got a version 4 uuid, the ids were distinct, pgcrypto was not installed, and 0 tables were left after ROLLBACK. gen_random_uuid() is built into PostgreSQL 13 and later (INFERRED from the PostgreSQL release notes, not

checked this session; CONFIRMED that it works on this 16.15 server without pgcrypto). That removes one extension to install. Random ids insert into the middle of an index instead of the end, which costs some speed at very large sizes (INFERRED; not measured). I judged that acceptable and keep one rule for all tables, including location_ping (decision 9).
Note. Q5 is open, and it points to chapter 10.
IDs. B6 (retry safe ids), B8 (not leaking volumes of personal data). Enforcers. column default gen_random_uuid() (database); ownership checks (application code only).

Question 6. Constraints

Constraints are covered in full in chapter 5, where every table is listed and the table of invalid states that are now impossible is set out. This chapter points to chapter 5 for the full treatment.

Question 7. Indexes

Indexes are covered in full in chapter 5, which lists each index that earns its place by a named query and the tables that are left without extra indexes on purpose. This chapter points to chapter 5 for the full treatment.

Summary. The seven answers

Question 1: each fact lives in exactly one home table and everything else finds it by a foreign key, with two deliberate denormalisations, the HISTORY snapshot and the SPEED cache.
Question 2: money is a whole number of kobo in a bigint column with a char(3) currency column beside it, always NGN, and only API validation can reject a decimal amount.
Question 3: status and state are enforced by the status_transition_rule table, triggers, partial unique indexes and check constraints.
Question 4: every table has created_at and updated_at, people are soft deleted while financial and audit rows are never deleted, and only the erasure function may blank the five personal text columns.
Question 5: every record id is a random UUID generated with gen_random_uuid(), and every endpoint must still check ownership.
Question 6: constraints are covered in full in chapter 5.
Question 7: indexes are covered in full in chapter 5.

Chapter 4. The state machine

docs/03_state_machine.md

Allowed trip transitions

A trip is Active (B0) in the statuses requested, assigned, arrived or started. Terminal means no further status change: completed and cancelled. Each allowed move below is also one row in status_transition_rule with entity = 'trip' (14 rows).

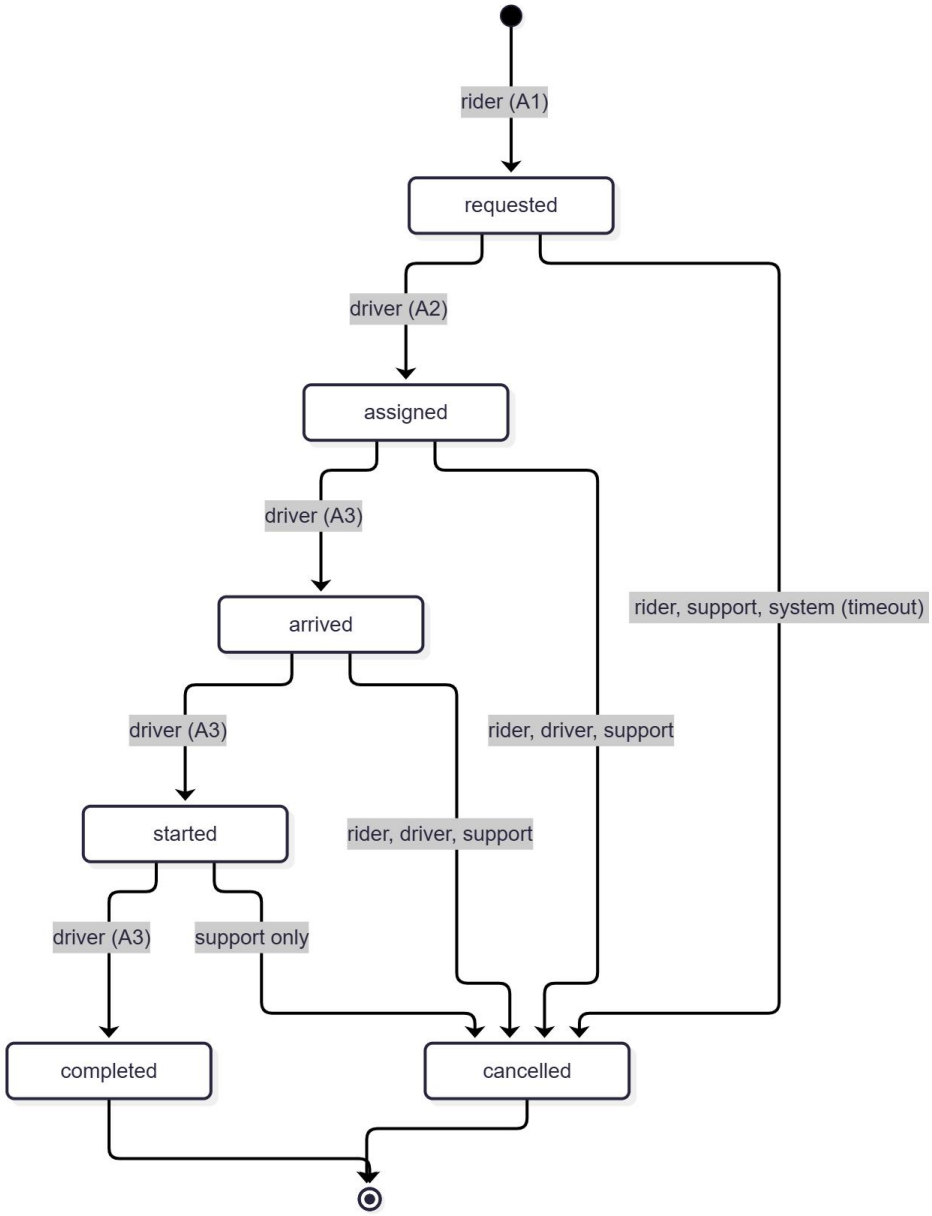


Image 4.1. The trip state machine

#	From	To	Who may do it	Serves	Meaning
T0	(new)	requested	rider	A1, B1, B5	The rider requests a trip. A new trip may only start as requested.
T1	requested	assigned	driver	A2, B2	A driver accepts. driver_id and vehicle_id are set in the same update.
T2	assigned	arrived	driver	A3	The driver reached the pickup.
T3	arrived	started	driver	A3	The rider is in the car. started_at is set.
T4	started	completed	driver	A3, B4	The drive is over. completed_at,

					fare_amount_minor, fare_currency and distance_m are set.
T5	requested	cancelled	rider, support	B9	No driver yet, so no fee. A driver cannot cancel here, because a requested trip has no driver (assumption 4).
T6	assigned	cancelled	rider, driver, support	B9	Fee only if the rider cancels.
T7	arrived	cancelled	rider, driver, support	B9	Fee only if the rider cancels.
T8	started	cancelled	support only	B9	Rider and driver cannot cancel once the ride has started.
T9	requested	cancelled	system	B9, B1	Timeout: no driver accepted within request_timeout_seconds (placeholder 600, needs owner approval). No fee. It frees the rider to request again (B1). The timer itself is application code (a scheduled job). The database only allows the move for actor system and only from requested.

Cancellation fee (B9, Q2). When status becomes cancelled, the database copies cancellation_fee_minor from the setting of the same name, but only when cancelled_by_role = 'rider' and the trip was already assigned or arrived. In every other cancellation, including the system timeout T9, the fee is 0 (assumption 6, decision 5). Open (Q6): if an assigned driver never arrives and the rider cancels, this rule still charges the rider, and the owner must decide before launch. The copy is a HISTORY snapshot, so a later change to the setting never changes an old trip.

Allowed payment transitions

A payment stays pending until the provider answers. It is never failed by our own clock. While a payment is pending, the one live payment rule (payment_one_live_per_purpose) keeps the rider from being charged twice, so a stuck pending payment blocks a retry (section 4b is the way out). Statuses: pending, succeeded, failed. Succeeded and failed are terminal. Rows in status_transition_rule with entity = 'payment': three (P1, P2, P3).

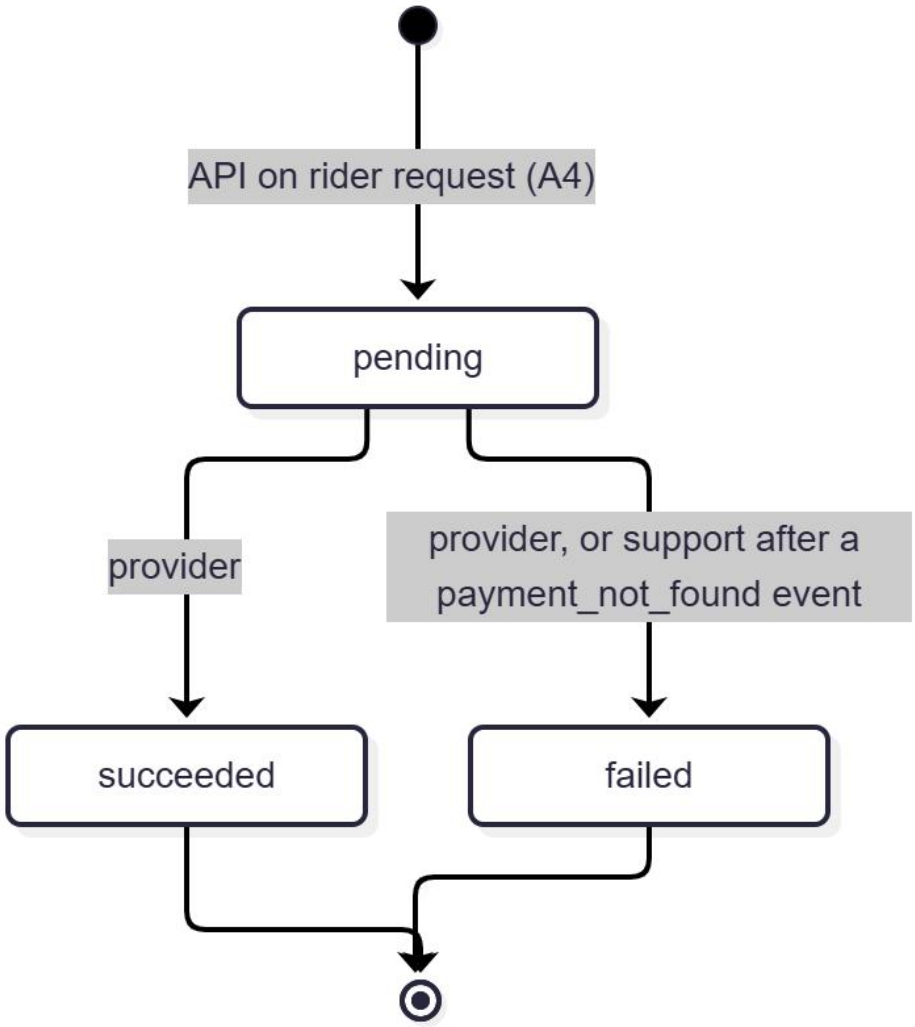


Image 4.2. The payment state machine

#	From	To	Who	Serves	Meaning
P0	(new)	pending	the API on the rider's request	A4, B6	A payment may only be created as pending, for a completed trip's fare or a cancelled trip's fee, with the exact amount owed
P1	pending	succeeded	provider	A4	Provider confirms. provider_reference and paid_at must be set
P2	pending	failed	provider	A4	Provider says declined. failure_reason must be set
P3	pending	failed	support	A4, B6	Only if a provider_event of type payment_not_found exists for this payment's provider_reference, was received after the payment was created, and is not older than the latest provider event of any type for that reference

					(a later payment_succeeded or payment_failed cancels the proof). failure_reason must be set. This is the way out of a stuck payment (section 4b). It is a person's decision after the provider said it has no record, not a timeout

Forbidden trip moves

A careless design would allow every row below. Each one is blocked. Three moves are shown in full first, then the rest follow in a compact table. Rows marked "tested" were run in a rolled back transaction (section 7); "yes, built database" names a test raised again on the finished database.

#	Forbidden move	Why it is dangerous	Blocked by	Tested
F3	A driver accepts a trip the rider cancelled a second ago (race)	Driver is sent to a rider who is gone. Not obvious: both requests are individually "valid"	the app's update says WHERE status = 'requested', and the trigger refuses cancelled to assigned	yes, step 14 (cut down copy, the trigger refuses cancelled to assigned); yes, built database, F3 race test (guard returns UPDATE 0, and without the guard RH005), evidence\stage5_export 4_f3_race.txt
F9	driver B moves driver A's trip to arrived	A stranger changes someone else's trip. Not obvious: the move itself is legal	actor check in trigger: driver_id must equal the actor's id	yes, step 7; yes, built database, E38 (migration_015_error_tests_compare.txt) and X2 (proofs_rejected_and_extra.txt)
F10	driver B takes a trip driver A already holds, without changing status	Not obvious: status stays "assigned", so the transition table is never consulted. This is the double accept bug	driver_id cannot change once set, in trip_enforce_transition	yes, step 6; yes, built database, E39 (RH006; migration_015_error_tests_compare.txt)

Move	Blocked by	Why
F1 completed to started (re open)	terminal freeze in trip_enforce_transition	changes a finished, possibly paid, trip (B4)
F2 cancelled to assigned (zombie trip)	terminal freeze in trip_enforce_transition	a cancelled trip comes back and gets a driver
F4 support cancels a completed trip	terminal freeze	hides the money trail (B4)
F5 requested to started (or any skipped step)	no matching row in status_transition_rule	history looks wrong (B5)
F6 started to arrived (going backwards)	no matching row	rewrites the trip story
F7 assigned to requested (a driver "un assigns")	no matching row	history looks as if the trip was never taken
F8 rider or driver cancels a started trip	no row for those roles	only support may (B9)
F11 a new trip inserted already as assigned or completed	insert check in trip_enforce_transition	skips A1 and A2
F12 rider marks the trip arrived	no row for that role	a rider cannot be the driver (A3)
F13 edit fare_amount_minor of a completed trip	terminal freeze	B4
F14 change rider_id or fare_quote_id of any trip	immutability check in trip_enforce_transition (RH015)	moves a trip to another account
F15 delete a trip row	trigger trip_block_delete	loses history (B4, B5)
F16 a driver has a second active trip	partial unique index trip_one_active_per_driver	B2
F17 a rider has a second active trip	partial unique index trip_one_active_per_rider	B1
F18 the system cancels an assigned, arrived or started trip	no row for actor system except requested to cancelled (T9)	cuts a rider or driver off mid ride
F19 the application inserts a history row or an audit row itself	ridehail_app has no INSERT on trip_status_history or audit_log (migration 010)	B5 would be meaningless

Step 15 of the test also showed the other direction: once a trip is cancelled or completed, the rider may request again (the partial index only covers active statuses).

Forbidden payment moves

Two moves are shown in full first, then the rest follow in a compact table.

#	Forbidden move	Why	Blocked by
PF2	failed to succeeded	Not obvious. The rider retried and a new payment exists. Flipping the old one could double charge (B6). A late success on a failed payment is handled as in section 4a	terminal freeze
PF10	support fails a pending payment with no payment_not_found event for it, or with an event for another reference, of another type, older than the payment, or superseded by a later event of any type	Support could write off a payment the provider actually took, with nothing to show why	the lookup inside trigger payment_enforce_rules (tested, S1 to S4)

Move	Blocked by	Why
PF1 succeeded to failed	terminal freeze in payment_enforce_rules	a late failed message must not erase money we hold
PF3 support or rider marks a payment succeeded	no row for those roles	only the provider can confirm money
PF4 a payment created already as succeeded	insert check in payment_enforce_rules	skips the provider
PF5 a fare payment for a trip that is not completed	trigger payment_enforce_rules (looks at trip.status)	charging before the ride ends
PF6 an amount different from the trip's fare_amount_minor or cancellation_fee_minor	trigger payment_enforce_rules	wrong charge (B3)
PF7 a second live payment (pending or succeeded) for the same trip and purpose	partial unique index payment_one_live_per_purpose (tested P2, P4)	double charge (A4, B6)
PF8 a refund on a payment that is not succeeded, or above the amount	trigger refund_enforce_total (tested R2, R4, R5)	B10
PF9 our own system marks a pending payment failed	no row for actor system in the payment rules (tested S9)	the provider may still have taken the money
PF11 support fails a succeeded payment, even when a payment_not_found event exists	terminal freeze (tested S7)	succeeded is terminal, money was taken
PF12 support marks a pending payment succeeded	no row for support to succeeded (tested S8)	only the provider can confirm money

Test ids in brackets (P2, P4, R2, R4, R5 and similar) refer to the payment test in the report, not to the payment moves P0 to P3 above. Step ids S1 to S11 belong to the stuck payment test in section 4b. Every payment status change is also written to audit_log by trigger payment_write_audit, so B5 covers payments (decision 6).

The rule for a stuck payment (move P3)

Why P3 needs a trigger. The condition looks at another table (provider_event). A check constraint sees only the payment row, so it cannot do this. The payment trigger payment_enforce_rules looks up the event.

```
P3 rule: Only if a provider_event of type payment_not_found exists for
this payment's provider_reference, was received after the payment was
created, and is not older than the latest provider event of any type for
that reference (a later payment_succeeded or payment_failed cancels the
proof). failure_reason must be set. This is the way out of a stuck
payment (section 4b). It is a person's decision after the provider said
it has no record, not a timeout.
```

The way out of a stuck pending payment

```
What "stuck" means. The payment is pending and the provider has not told
us the result. By payment_one_live_per_purpose the rider cannot start
another payment for the same fare, so the rider is blocked (A4, B6).

Reconciliation job (application code). Setting
payment_reconcile_after_seconds (placeholder 900, needs owner approval).
Any payment that has been pending longer than that is checked with the
provider by its provider_reference. The job does one of these, in one
database transaction:
```

Test of the trigger idea

I built a cut down copy of the trip table, the rule table and the trigger inside BEGIN ... ROLLBACK, with the same logic as trip_enforce_transition and the two partial indexes. After the rollback I counted tables and functions: 0 tables, 0 functions (CONFIRMED). The history table received 9 rows from the trigger: 4 creation rows (from_status empty), 4 moves of trip 1 (assigned, arrived, started, completed) and 1 cancellation, each with the right actor role. Not tested: two sessions racing at the same moment, the trip_block_delete trigger, the cancellation fee copy, and the snapshot copy. The new system move T9 was not run (it uses the same lookup as the tested moves).

Scheduled jobs

Two rules need a clock or a poller, and the database cannot run one by itself: the request timeout job (T9, B9, B1) and the payment reconciliation job (section 4b, A4, B6). Both jobs are application code. They are not designed or built here. The request timeout job finds trips with status requested older than request_timeout_seconds (placeholder 600, needs owner approval) and cancels each one with actor system, no fee. The payment reconciliation job finds payments pending longer than payment_reconcile_after_seconds (placeholder 900, needs owner approval) and checks each with the provider, then acts as in the table in 4b. Both connect as role ridehail_app, never the dev superuser ridehail, and both are safe to run twice. Each interval is a setting, and Stage 4 applied placeholders of 60 and 300 seconds that the owner must confirm (09_open_items.md). If either job is down for long, the owner should be told; a health signal for the jobs (last successful run time) is not designed here.

Chapter 5. Constraints and indexes

Built from docs/04_hard_questions.md (questions 6 and 7), evidence/catalogue_check_016.txt, db/migrations/004_quotes_trips.sql (lines 108 to 111), db/migrations/005_money.sql (lines 30 to 31), db/migrations/016_payment_trip_id_index.sql, and evidence/migration_016_measure_before.txt and _after.txt (the P1 plan node lines only).

Invalid states that are now impossible

First every table, then the table of invalid states that are now impossible. "Tested" points to a rolled back run. The ids are: steps 1 to 16 are the trigger test in 03_state_machine.md section 7; P, R, I and G ids are the payment, refund, idempotency and rating tests of the Stage 2 report; E ids are the provider_event test in 03 section 4a; X ids are the erasure test in question 4; S ids are the stuck payment test in 03 section 4b (test 4 of the round 2 report). "Not tested" means I reasoned it only.

Invalid state	ID	Exact blocker	Enforced by	Tested
A rider with two active trips	B1	partial unique index trip_one_active_per_rider	database	yes, trigger test step 2
A driver with two active trips	B2	partial unique index trip_one_active_per_driver	database	yes, step 5
A rating without a completed trip	B7	trigger rating_enforce_rules	database	yes, G4
A rating after the 7 days	B7	trigger rating_enforce_rules	database	yes, G5 (test used a fixed 7 days; the real trigger reads the setting)
A second rating by the same person on a trip	B7	unique constraint rating_one_per_role_per_trip	database	yes, G2
Two payments for one trip fare	A4, B6	partial unique index payment_one_live_per_purpose	database	yes, P2 and P4
A duplicate idempotency key	B6	unique constraint idem_one_key_per_rider_action	database (the replay of the stored answer is application code)	yes, I1; I2 and I3 show it is per rider and per action
A duplicate provider payment reference	B6	unique constraint payment_provider_reference_key	database	yes, P8
A negative or zero payment	B3	payment_amount_positive	database	yes, P5 and P6
Negative amounts elsewhere	B3	quote_amount_not_negative, trip_amounts_not_negative, refund_amount_positive, setting_value_not_negative	database	not tested (same kind of check as P5)
A currency other than NGN	B3	payment_currency_ngn, quote_currency_ngn, trip_currency_ngn, refund_currency_ngn, setting_currency_ngn	database	yes for payment (P7); the others not tested
Editing a completed trip's fare	B4	terminal freeze in trip_enforce_transition	database	yes, step 12
A completed trip with no fare	B4	trip_completed_has_facts	database	not tested at first; later yes on the built database, E42 (SQLSTATE 23514, "violates check constraint "trip_completed_has_facts"", evidence/migration_015_error_tests_compare.txt line 60)
Refunds above the payment	B10	trigger refund_enforce_total	database	yes, R2 and R4; R5 for a non succeeded payment
A trip using another driver's vehicle	A2	trip_vehicle_belongs_to_driver	database	not tested at first; later yes on the built database, E09 (SQLSTATE 23503, "violates foreign key constraint "trip_vehicle_belongs_to_driver"", same file, line 12)
A trip using another rider's quote	A1	trip_quote_belongs_to_rider	database	not tested at first; later yes on the built database, E02a and E02b (SQLSTATE 23503, "violates foreign key constraint "trip_quote_belongs_to_rider"", same file, lines 3 and

				4)
A trip started on an expired quote	A1	trigger trip_enforce_transition (insert branch)	database	not tested at first; later yes on the built database, E04 (SQLSTATE RH004, "fare quote ... has expired", same file, line 6)
A location ping on a finished trip	A3	trigger ping_enforce_trip_status	database	not tested at first; later yes on the built database, E10 (SQLSTATE RH030, "a location ping needs a trip that is assigned, arrived or started (status is completed)", same file, line 13)
An illegal status move	B5	status_transition_rule + trip_enforce_transition	database	yes, steps 3, 6 to 8, 10, 11, 14, 16
A status change with no history row	B5	trigger trip_write_history	database	yes (9 history rows in the test)
A decimal amount such as 10.5 kobo	B3	API validation	application code only	confirmed that the database does not stop it (question 2)
A person reading or changing someone else's trip	B8	ownership check on every endpoint	application code only (the trigger check on the claimed actor id helps, but only trusts what the app says)	not applicable yet
A retry that gets a new answer instead of the stored one	B6	code that looks up and returns response_body	application code only	not applicable yet
Support failing a pending payment with no proof, with the wrong reference, the wrong type, or an old event	A4, B6	trigger payment_enforce_rules (move P3 lookup)	database	yes, test 4 S1 to S4
Support failing a succeeded payment, or marking a pending one succeeded	A4, B6	terminal freeze and no rule row for that move	database	yes, test 4 S7 and S8
A provider callback applied twice	U4, B6	unique constraint provider_event_provider_event_id_key (the second message is refused or does nothing)	database (the handler that decides outcomes is application code)	yes, test 1 steps E1 to E3b (one row after repeats)
Our own clock marking a pending payment as failed	A4	no row for actor system in the payment rules, and trigger payment_enforce_rules	database	not tested (same lookup as the trip moves)
The API role blanking an address or editing a rating comment directly	B4, B8	the freeze trigger and rating_block_change check current_user	database	yes, test 2 steps X1 to X3
A role with no grant calling the erasure function	B8	EXECUTE revoked from PUBLIC and granted only to ridehail_app	database	yes, test 2 step X7
Erasing a person with an active trip	B8	check inside erase_rider and erase_driver	database (the driver version was untested at first)	yes for riders, test 2 step X6; for drivers, later on the built database: the refusal is E26b (SQLSTATE RH071, "driver ... has an active trip: cannot erase (B8)", evidence/migration_015_error_tests_compare.txt line 31) and erase_driver itself ran in X6e (evidence/proofs_rejected_and_extra.txt, line 256)
A request cancelled by a non system actor for timeout, or the system cancelling an assigned trip	B9	rule rows in status_transition_rule	database	not tested
A timeout job that never runs	B9, B1	the scheduled job	application code only	not applicable yet

The catalogue check

The catalogue check and three passes after migration 016 and the document updates (2026-10-07T15:48Z) compared the documents with the database: tables 16, constraints 116, indexes 42, triggers 27, functions 18 and roles 4, with total mismatches 0. Pass 2 counted 22 foreign key rows and 66 check rows, and the index count from documents as 32 constraint backed plus 10 chosen, which is 42.

Object	Count
Tables	16
Constraints	116
Indexes	42
Triggers	27
Functions	18
Roles	4

Per entity constraints: payment as the model

Kind	Name	Rule	Index or justification
PK	payment_pkey	id	own index
UNIQUE	payment_provider_reference_key	provider_reference, which is a required column: a payment row exists only after the provider returned its reference, so every payment can be matched with provider_event rows	own index
UNIQUE (partial)	payment_one_live_per_purpose	(trip_id, purpose) where status is pending or succeeded	own index
FK	payment_trip_id_fkey	trip_id -> trip.id	index payment_trip_id_idx (migration 016, Stage 4: before it, this column had no index of its own; payment_one_live_per_purpose is partial and covers live payments only)
CHECK	payment_purpose_valid	fare or cancellation_fee	none needed
CHECK	payment_status_valid	pending, succeeded or failed	none needed
CHECK	payment_amount_positive	amount_minor above 0	none needed
CHECK	payment_currency_ngn	currency is NGN	none needed
CHECK	payment_succeeded_has_proof	succeeded needs provider_reference and paid_at; paid_at only when succeeded	none needed
CHECK	payment_failed_has_reason	failed needs failure_reason, and only then	none needed
Trigger	payment_enforce_rules	insert: pending only, by the trip rider (claimed actor), for a completed trip fare or a cancelled trip fee, exact amount. Terminal rows: only updated_at may change. Pending rows: a status change needs a rule row for the claimed actor and only the columns of the move may change. For move P3 (support fails a pending payment): a payment_not_found event for the same provider_reference, received after the payment was created AND not older than the latest provider event of ANY type for that reference (a later payment_succeeded cancels the proof). Proved on the real database (X5a to X5c)	uses index provider_event_reference_idx for that lookup
Trigger	payment_write_audit	after a status change, writes	n/a

		audit_log (status_change, from and to) with the claimed actor. No row at creation. SECURITY DEFINER, owner ridehail_writer. EXECUTE revoked from PUBLIC (migration 012); search_path ends with pg_temp (migration 013)	
Trigger	set_updated_at		n/a

The other 15 per entity tables are in docs/04_hard_questions.md and listed in the appendix by name.

Indexes

I add few indexes beyond the ones constraints already create. A new index makes every write slower and takes space, so each one below earns its place by a named query. Ten are created on purpose.

Index	Table	Columns	Query it serves
trip_requested_created_at_idx	trip	(created_at) WHERE status = 'requested'	A2 open requests near a driver and the request timeout job; a partial index holds only waiting trips
vehicle_driver_id_idx	vehicle	(driver_id)	A2 find the driver's vehicle; also supports vehicle_driver_id_fkey
trip_status_history_trip_seq_idx	trip_status_history	(trip_id, seq)	A3 status timeline ordered by seq; also supports history_trip_id_fkey
location_ping_trip_recorded_idx	location_ping	(trip_id, recorded_at DESC)	A3 latest driver position for the rider's map; also supports ping_trip_id_fkey
refund_payment_id_idx	refund	(payment_id)	the refund total check (B10); also supports refund_payment_id_fkey
trip_rider_history_idx	trip	(rider_id, created_at DESC, id DESC)	rider history list pages with keyset pagination; also supports trip_rider_id_fkey
trip_driver_history_idx	trip	(driver_id, created_at DESC, id DESC)	driver history list pages with keyset pagination; also supports trip_driver_id_fkey
payment_pending_created_at_idx	payment	(created_at) WHERE status = 'pending'	reconciliation job for pending payments older than a setting
provider_event_reference_idx	provider_event	(provider_reference)	the P3 lookup in payment_enforce_rules
payment_trip_id_idx	payment	(trip_id)	EP03 every payment of a trip (any status, so a failed attempt stays visible), EP15 and the support payment views

The other 32 indexes exist because a constraint made them: PK and UNIQUE constraints create their own index.

The statements that enforce the rules

B0, B1, B2: ACTIVE means requested, assigned, arrived, started. A trip with no driver has an empty driver_id, which never collides in the driver index, so it counts for the rider only. A partial unique index makes two active trips impossible, one per rider and one per driver.

```
CREATE UNIQUE INDEX trip_one_active_per_rider ON trip (rider_id)
  WHERE status IN ('requested','assigned','arrived','started');
CREATE UNIQUE INDEX trip_one_active_per_driver ON trip (driver_id)
  WHERE status IN ('requested','assigned','arrived','started');
```

A4, B6: one live (pending or succeeded) payment per trip and purpose. A failed one does not block a retry.

```
CREATE UNIQUE INDEX payment_one_live_per_purpose ON payment (trip_id, purpose)
  WHERE status IN ('pending','succeeded');
```

An index added in Stage 4: the payments of a trip

In migration 016 an ordinary index was added on payment (trip_id), serving A4, B6, U1, U3 (EP03 payments of a trip, EP15 payment lookup per trip, support payment views). The only index that starts with trip_id on payment is payment_one_live_per_purpose, which is PARTIAL (pending and succeeded only), so a read of every payment of a

trip (any status) could not use it and was a parallel sequential scan of the whole table (169,410 rows); the foreign key payment_trip_id_fkey also had no index on its referencing column.

```
CREATE INDEX payment_trip_id_idx ON payment (trip_id);
```

Before migration 016 the P1 read of all payments of a trip (every status), run 1, was:

```
 Gather (actual time=9.595..17.314 rows=1 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   Buffers: shared hit=3427
   ->  Parallel Seq Scan on payment (actual time=8.226..9.490 rows=0 loops=3)
         Filter: (trip_id = '40000000-0000-0000-0000-000000198904'::uuid)
         Rows Removed by Filter: 56470
```

After migration 016 the same read, run 1, is:

```
 Index Scan using payment_trip_id_idx on payment (actual time=0.073..0.074 rows=1 loops=1)
   Index Cond: (trip_id = '40000000-0000-0000-0000-000000198904'::uuid)
   Buffers: shared hit=1 read=3
```

The list of migrations 001 to 016 by file name is in Appendix B.

Chapter 6. API contracts

Built from docs/06_api_contracts.md, docs/06b_error_catalogue.md and docs/06c_endpoint_matrix.md.

Conventions

Contracts only: no code, no database change. A second engineer must be able to build every endpoint from the contract and the error catalogue without asking a question. Field names are the column names of 02_entities.md, statuses are those of 03_state_machine.md, and every endpoint cites requirement IDs from 01_requirements.md. CONFIRMED means the rule was run and the output kept; INFERRED means it was reasoned.

Format

Base path /v1. Requests and responses are JSON, UTF-8, Content Type application/json (otherwise 415). The body is at most 16 KiB (otherwise 413). Unknown body fields are rejected (422 validation_failed, the field is named): the caller must not be able to send rater_role, status, amount_minor or driver_id and have them used. Ids are uuids. Timestamps are RFC 3339 UTC with microseconds (2026-09-16T23:53:32.458283Z). Money is an integer number of kobo in a field ending _minor, with a currency field beside it, always "NGN" (B3); a number that is not an integer (for example 10.5) is rejected with 422, because the database would round it silently (10.5 became 11, CONFIRMED in 04 question 2), and one amount is at most 1,000,000,000 kobo. Coordinates are JSON numbers with at most 6 decimals (the columns are numeric(8,6) and numeric(9,6)); latitude is -90 to 90, longitude is -180 to 180. Lists use the envelope {"items": [...], "next_cursor": "..." or null} (section 1.9).

Who calls

A bearer token (Authorization: Bearer <token>) gives the API a role and a subject id (sub, a uuid); the API trusts only these two values from the token.

Caller	How it is identified	sub is
rider	bearer token, role rider	rider.id
driver	bearer token, role driver	driver.id
support	bearer token, role support	support_admin.id
provider	no token. HMAC signature of the body (section EP19)	none
scheduler	internal process, no HTTP (section 5)	none

A token of a person whose deleted_at is set, or of a support admin with is_active = false, is refused with 401 (INFERRED: the database does not check this).

Ownership checks

Every endpoint that takes an id compares a column with sub, in the same SQL statement as the read or write (... WHERE id = $1 AND rider_id = $2), so there is no gap between the check and the use.
A failed ownership check returns 404 not_found with the same body as an id that does not exist (never 403), so a stranger cannot learn that an id exists. A role that may never call an endpoint gets 403 forbidden_role. The ownership column for a driver is trip.driver_id; while a trip is requested it is NULL and equals no caller, so a driver cannot touch a trip it has not accepted.

Session settings the API sets for the database

Every request runs in one database transaction on the role ridehail_app (never the dev superuser). The first statement of the transaction is:

```
SELECT set_config('app.actor_role', $1, true), set_config('app.actor_id', $2, true);
```

The value comes only from the authenticated caller: $1 is the token role (rider, driver, support) and $2 is sub; for the webhook it is provider and the empty string, for scheduled jobs system and the empty string, never from the request body, a header or a query string. The third argument is true (transaction local), because with a connection pool the same database connection serves different callers and a session level value would leak to the next caller. The database trusts the claim: it checks that the claimed actor may make the move and that the claimed id is the trip's own rider or driver, but it cannot prove who the caller is, which is the API's job.

The error body, and what never leaves the server

The body is:

```
{"error": {"code": "rider_has_active_trip", "message": "You already have a trip in progress.",
           "request_id": "req_01J8...",
           "details": {"current_status": "requested"}}}
```

- code is machine readable and comes from 06b_error_catalogue.md. message is a fixed plain English sentence per code. details holds only safe, named fields.
- A 500 never carries a database message, because a Postgres message can contain ids and values (CONFIRMED). A 500 body has code: internal_error, the fixed message and the request_id, nothing else; the real error goes to the server log with the same request_id.

• Every response carries X Request Id. The caller may send its own (up to 64 characters); otherwise the API makes one.

How a database error becomes an HTTP status

The database driver gives the API the SQLSTATE, the constraint name (for 23xxx errors) and the message. The API reads the SQLSTATE (and, for 23xxx, the constraint name) and never parses the message text, which is for the server log only. It maps in this order: 23505, 23503, 23514 (unique, foreign key, check) by the constraint name (for example trip_one_active_per_rider); RHnnn (a trigger or function RAISE EXCEPTION, migration 015) by the SQLSTATE alone, since every rule has its own code in the custom class RH and the table of 45 codes is in 06b section 3, while a SQLSTATE in class RH that is not in the table, and the old generic P0001, are internal_error plus an alert, never a guess; 55P03 (lock timeout) on the idempotency insert is request_in_progress, and 40001 or 40P01 (serialization or deadlock) retry the whole transaction once, then service_unavailable; 42501 (permission denied), 23502 (not null) and anything else are internal_error (a role or API bug). The API reads the SQLSTATE and the constraint name in exactly one place, a database error mapper that every request goes through, and nowhere else; the field name differs between drivers, so the mapper is the only code that must change when the driver is chosen.

Idempotency in this API

Three mechanisms; each endpoint says which one it uses.

• (a) Keyed: the Idempotency Key header. Only two rider actions use the table idempotency_key: POST /v1/trips (action request_trip) and POST /v1/trips/{trip_id}/payments (action pay_trip). Support refunds also take the header but cannot use the table (its rider_id is required), see EP21.

• (b) State based replay for trip moves. accept, arrive, start, complete and cancel have no key; a repeated move by the caller who already made it returns 200 with the current trip and Idempotent Replay: true, and anything else is 409.

• (c) Natural idempotency. Reads. Erasure (running it twice is harmless). Ratings are not keyed: a second rating by the same side is 409 already_rated. Location pings and support notes are not idempotent: a duplicate ping or note is harmless.

For the keyed mechanism, the key is 1 to 128 printable ASCII characters without spaces, otherwise 400 invalid_idempotency_key. The unique constraint is idem_one_key_per_rider_action on (rider_id, action, key): a key is unique per rider and action, not global. The first statement after the settings inserts the key with expires_at = now + 24 hours. Only successful (2xx) responses are stored. Same key with a different body is 422 idempotency_key_reuse; the same key still running is 409 request_in_progress with Retry After: 1.

The three transactions of EP15 (the payment flow) show why a provider call must not hold a database lock:

```
Flow (three transactions, because a provider call must not hold a database lock):
1. Transaction 1: set the actor; INSERT INTO idempotency_key ... ON CONFLICT DO NOTHING
   (action pay_trip, request_hash includes the trip_id); commit. Replay rules of 1.7a apply here.
2. Call the provider to start the charge, passing our Idempotency-Key as the provider's own
   idempotency key (assumption, decision 5). The provider returns a provider_reference.
   Provider unreachable or refusing: delete the key row, 502 provider_error, no payment row exists.
3. Transaction 2: set the actor; INSERT INTO payment (id, trip_id, purpose, amount_minor, currency,
   provider_reference) (status pending); UPDATE idempotency_key SET response_status = 202,
   response_body = ..., payment_id = ...; commit.
```

A worked endpoint

EP02 POST /v1/trips is the worked endpoint because it shows the key, the transaction and the errors (rider app; A1, B0, B1, B6).

Who and ownership: role rider. The trip's rider_id is sub. The quote must satisfy fare_quote.id = body.fare_quote_id AND fare_quote.rider_id = sub.

Request: headers Authorization, Content Type, Idempotency Key (required). Body:

Field	Type	Required	Limits
fare_quote_id	uuid	yes	a quote of this rider that has not expired and has not started a trip
pickup_address	string	yes	1 to 300 characters, not blank
dropoff_address	string	yes	1 to 300 characters, not blank

Flow (one transaction): set the actor; insert the idempotency row; read the quote with the ownership condition; INSERT INTO trip (id, rider_id, fare_quote_id, pickup_address, dropoff_address) (status defaults to requested; the trigger writes the history row (new) > requested); UPDATE idempotency_key SET response_status = 201, response_body = ..., trip_id = ... CONFIRMED in the walkthrough, evidence/api_walkthrough_tests.txt.

Success: 201, header Location: /v1/trips/{id}, body Trip (rider view, status = requested). A replay returns the same 201 body with Idempotent Replay: true.

Errors:

Status	Code	When	Database cause
409	rider_has_active_trip	the rider already has a trip in requested, assigned, arrived or started	unique index trip_one_active_per_rider, "duplicate key value violates unique constraint trip_one_active_per_rider" (E01)
404	quote_not_found	no quote with that id for this rider (unknown id, or another rider's quote)	ownership query; backstop FK trip_quote_belongs_to_rider, "insert or update on table trip violates foreign key constraint trip_quote_belongs_to_rider" (E02a, E02b)
409	quote_already_used	the quote already started a trip	unique trip_fare_quote_id_key (E03)
422	quote_expired	the quote's expires_at has passed	trigger trip_enforce_transition, SQLSTATE RH004 (message "fare quote % has expired", E04); the API also checks expires_at first
400	invalid_idempotency_key	header missing, empty, longer than 128, or has a space	backstop check idem_key_length (E32)
422	idempotency_key_reuse	the key was used with a different body	none: the stored request_hash differs
409	request_in_progress	the same key is still being processed	SQLSTATE 55P03 on the idempotency insert (concurrency test)
422	validation_failed	an address is blank or too long, fare_quote_id is not a uuid, a field is unknown	none

Also: 401, 403, 429, 500, 503 (common). Order of checks: replay and key checks come first, so a retry with the same key never returns rider_has_active_trip (it replays the 201). A new key after a success does return it. Idempotency: keyed (section 1.7a), action request_trip. A retry cannot create a second trip: the stored answer is replayed, and even without it trip_one_active_per_rider and trip_fare_quote_id_key refuse a second trip. Session settings: rider, sub.

The list contract

EP06 GET /v1/riders/me/trips (rider app; A3, U1, B8) is the worked list endpoint. The list is WHERE rider_id = sub; there is no rider_id parameter. Query parameters are limit, cursor, status, created_from and created_to.
Sort: created_at descending, ties broken by id descending. Nothing else is offered. Reason: the indexes are trip_rider_history_idx (rider_id, created_at DESC, id DESC) and trip_driver_history_idx (driver_id, created_at DESC, id DESC). Any other sort needs a sort of all of a person's trips and breaks keyset paging.
Keyset, not OFFSET. The query asks for limit + 1 rows; if the extra row exists there is a next page and next_cursor is built from the last item returned. Page size: limit default 20, minimum 1, maximum 100. Outside that range: 400 invalid_filter (the field limit is named).
The cursor is opaque to the client. It is base64url(payload) + "." + base64url(signature). The payload is JSON {"v":1,"c":"<created_at of the last item>","i":"<id of the last item>","f":"<16 hex of SHA-256 of the caller and the filters>"}. The signature is HMAC SHA256 of the payload with a server secret read from the environment. The API refuses a cursor when the signature does not verify, v is not 1, f differs from the current caller and filters, or c or i do not parse: 400 invalid_cursor. Binding f means a cursor cannot be moved to another caller or reused with other filters.
Filters (rider and driver lists): status (one or more of requested, assigned, arrived, started, completed, cancelled, comma separated), created_from and created_to (RFC 3339; created_at >= created_from and created_at < created_to). Any other query parameter, an unknown status, a bad date, or created_from not before created_to: 400 invalid_filter. A page after the last returns {"items": [], "next_cursor": null} with 200.
Worked example (real rows of rider 10000000-0000-0000-0000-000000000777, who has 40 trips; the signature is masked). The request is GET /v1/riders/me/trips?limit=3, and the first item with the cursor is:


```
{"items": [
  {"id": "40000000-0000-0000-0000-000000198904", "status": "completed", "pickup_address": "BULK pickup 198904",
"dropoff_address": "BULK dropoff 198904", "driver_name_snapshot": "BULK Driver 217", "created_at": "2026-10-
```

04T08:33:32.458283Z", "completed_at": "2026-10-06T18:11:00.519301Z", "cancelled_at": null, "fare_amount_minor": 108000, "fare_currency": "NGN", "cancellation_fee_minor": null, "cancellation_fee_currency": null}], "next_cursor": "eyJ2IjoxLCJjIjoiMjAyNi0wOS0xNlQyMzo1MzozMi40NTgyODNaIiwiaSI6IjQwMDAwMDAwLTAwMDAtMDAwMC0wMDAwLTAwMDAwMDE4ODkwNCIsImYiOiJkMzdjMGVjNmRkNWQzM2RmIn0.****"}

The cursor payload decodes to {"v":1,"c":"2026-09-16T23:53:32.458283Z","i":"40000000-0000-0000-0000-000000188904","f":"d37c0ec6dd5d33df"}. The database returned 4 rows for limit + 1, and the fourth (...183904) is why a next page exists (CONFIRMED, L2).

EP07 GET /v1/drivers/me/trips (driver app; A3, U2, B8) is identical to EP06 with these differences: role driver; WHERE driver_id = sub; no driver_name_snapshot; index trip_driver_history_idx; the item has no cancellation_fee_minor and no cancellation_fee_currency. EP08 is a bounded list that always sends next_cursor: null, and EP04, EP18 and EP22 send no cursor because they are small.

The endpoint matrix

The endpoint matrix is generated from the endpoint index by a script, so the two cannot disagree. Part 1 below is the endpoints against caller roles (Y = may call; the ownership check for each is in section 1.3 of the contract).

Id	Method and path	rider	driver	support	provider	scheduler
EP01	POST /v1/fare quotes	Y	-	-	-	-
EP02	POST /v1/trips	Y	-	-	-	-
EP03	GET /v1/trips/{trip_id}	Y	Y	Y	-	-
EP04	GET /v1/trips/{trip_id}/status history	Y	Y	Y	-	-
EP05	GET /v1/trips/{trip_id}/driver location	Y	-	Y	-	-
EP06	GET /v1/riders/me/trips	Y	-	-	-	-
EP07	GET /v1/drivers/me/trips	-	Y	-	-	-
EP08	GET /v1/trip requests	-	Y	-	-	-
EP09	POST /v1/trips/{trip_id}/accept	-	Y	-	-	-
EP10	POST /v1/trips/{trip_id}/arrive	-	Y	-	-	-
EP11	POST /v1/trips/{trip_id}/start	-	Y	-	-	-
EP12	POST /v1/trips/{trip_id}/complete	-	Y	-	-	-
EP13	POST /v1/trips/{trip_id}/locations	-	Y	-	-	-
EP14	POST /v1/trips/{trip_id}/cancel	Y	Y	Y	-	-
EP15	POST /v1/trips/{trip_id}/payments	Y	-	-	-	-
EP16	GET /v1/payments/{payment_id}	Y	-	Y	-	-
EP17	POST /v1/trips/{trip_id}/ratings	Y	Y	-	-	-

EP18	GET /v1/trips/{trip_id}/ratings	Y	Y	Y	-	-
EP19	POST /v1/provider/webhooks	-	-	-	Y	-
EP20	POST /v1/support/payments/{payment_id}/fail	-	-	Y	-	-
EP21	POST /v1/support/payments/{payment_id}/refunds	-	-	Y	-	-
EP22	GET /v1/support/settings	-	-	Y	-	-
EP23	PUT /v1/support/settings/{key}	-	-	Y	-	-
EP24	POST /v1/support/notes	-	-	Y	-	-
EP25	POST /v1/support/riders/{rider_id}/erasure	-	-	Y	-	-
EP26	POST /v1/support/drivers/{driver_id}/erasure	-	-	Y	-	-
EP27	GET /v1/trips/{trip_id}/events	Y	-	Y	-	-
JB1	request timeout job (internal)	-	-	-	-	Y
JB2	payment reconciliation job (internal)	-	-	-	-	Y
JB3	idempotency cleanup job (internal)	-	-	-	-	Y

The endpoint index gives every endpoint with what it does and its requirement IDs:

Id	Method and path	What it does
EP01	POST /v1/fare quotes	request a fare quote (A1, B3, Q1)
EP02	POST /v1/trips	request a trip (A1, B0, B1, B6)
EP03	GET /v1/trips/{trip_id}	read one trip (A3, B4, U1, U2, U3)
EP04	GET /v1/trips/{trip_id}/status history	read the trip's status history (A3, B5, U3)
EP05	GET /v1/trips/{trip_id}/driver location	read the driver's latest position (A3, B8)
EP06	GET /v1/riders/me/trips	list the rider's own trips (A3, U1, B8)
EP07	GET /v1/drivers/me/trips	list the driver's own trips (A3, U2, B8)
EP08	GET /v1/trip requests	list open trip requests near the driver (A2, U2)
EP09	POST /v1/trips/{trip_id}/accept	accept a trip (A2, B2, U2)
EP10	POST /v1/trips/{trip_id}/arrive	mark the driver arrived (A3, B5)
EP11	POST /v1/trips/{trip_id}/start	start the trip (A3, B5)
EP12	POST /v1/trips/{trip_id}/complete	complete the trip and set the fare (A3, B3, B4)
EP13	POST /v1/trips/{trip_id}/locations	record a location ping (A3, B8)
EP14	POST /v1/trips/{trip_id}/cancel	cancel the trip (B9, B1, B2, B5, Q2, U3)
EP15	POST /v1/trips/{trip_id}/payments	start a payment (A4, B3, B6, U1)
EP16	GET /v1/payments/{payment_id}	read one payment (A4, U3)
EP17	POST /v1/trips/{trip_id}/ratings	rate the trip (A5, B7, Q3)
EP18	GET /v1/trips/{trip_id}/ratings	read the trip's ratings (A5, B7)

EP19	POST /v1/provider/webhooks	receive a provider webhook (A4, B5, B6, U4)
EP20	POST /v1/support/payments/{payment_id}/fail	support marks a payment failed (A4, B6, U3, U4)
EP21	POST /v1/support/payments/{payment_id}/refunds	support refunds a payment (B10, B3, B4, U3, Q4)
EP22	GET /v1/support/settings	list the settings (B9, U3)
EP23	PUT /v1/support/settings/{key}	set one setting (B3, B7, B9, U3, Q1, Q2, Q3)
EP24	POST /v1/support/notes	add a support note (B5, U3)
EP25	POST /v1/support/riders/{rider_id}/erasure	erase a rider (B8, U3)
EP26	POST /v1/support/drivers/{driver_id}/erasure	erase a driver (B8, U3)
EP27	GET /v1/trips/{trip_id}/events	stream one trip's events (A3, B8, U1, U3)
JB1	request timeout job (internal)	cancel stale requested trips (B9, B1)
JB2	payment reconciliation job (internal)	reconcile pending payments (A4, B6, U4)
JB3	idempotency cleanup job (internal)	delete expired idempotency keys (B6)

Two requirement IDs have no endpoint: Q5 Location retention (no retention period is chosen, blocked on legal advice) and Q6 Driver never arrives (an open owner decision; EP14 applies the current rule and says so). No endpoint has no requirement ID.

The error catalogue

The body is always {"error": {"code", "message", "request_id", "details"}}. The mapping uses the SQLSTATE only: by constraint name for 23xxx, by the RHnnn code for every trigger and function rule. A SQLSTATE not in this catalogue, including the old generic P0001, is internal_error plus an alert, never a guess. Retryable means the same request may be repeated unchanged and can succeed later.

The 12 most important codes:

Code	**HTTP**	**Meaning**
rider_has_active_trip	409	rider already has a requested, assigned, arrived or started trip
quote_expired	422	the quote's expires_at has passed
illegal_transition	409	the move is not allowed from the trip's current status for this actor
trip_finished	409	the trip is completed or cancelled
payment_already_exists	409	a pending or succeeded payment already exists for this trip and purpose
refund_exceeds_payment	409	refunds would exceed the payment amount (B10)
already_rated	409	this side already rated the trip
idempotency_key_reuse	422	same key with a different body
request_in_progress	409	the same idempotency key is still being processed
invalid_cursor	400	cursor signature, version, filter fingerprint or fields invalid; or a Last Event ID / last_seq (EP27) that is not a whole number of 0 or more
rate_limited	429	too many requests
internal_error	500	any unexpected error, including every RH code marked internal_error in section 3

The other 33 codes, and the full table of the 45 SQLSTATE codes of the custom class RH from migration 015, are in docs/06b_error_catalogue.md sections 1 and 3 and are referenced in the appendix.

The full text of each of the 27 endpoints stays in docs/06_api_contracts.md section 4 and the three internal jobs JB1 to JB3 likewise in section 5.

Chapter 7. Over fetching

docs/07_over_fetching.md

The endpoint and why

EP03 GET /v1/trips/{trip_id} (trip detail) is the one endpoint that gathers a trip row, the quote (coordinates, estimate), the driver and vehicle snapshot, and the payments, for three clients that need very different parts of it: the rider app, the driver app and the support console. It is also the endpoint the rider's app would poll to watch a trip (A3), so its size matters for every poll. The trip list (EP06) is used as a second measurement in section 3 because the rider's list row is the most common client need and it exposes a gap in the contract (see section 3, finding 1). What EP03 does not carry already limits the damage: the status history (EP04), the ratings (EP18) and the payment detail with refunds and provider events (EP16, support view) are separate endpoints. EP03 is the "medium" document, not an "everything" document.

The real response and its size

Rider view of EP03 for the completed trip eeeeeeee-0000-0000-0000-000000000001 of the small seed (CONFIRMED, evidence/part_b_over_fetch_measurements.txt, M2; same values, keys reordered to the contract order for reading; the database returns keys in its own order, which does not matter to a client):


```
{
  "id": "eeeeeeee-0000-0000-0000-000000000001",
  "status": "completed",
  "rider_id": "aaaaaaaa-0000-0000-0000-000000000001",
  "fare_quote_id": "ffffffff-0000-0000-0000-000000000001",
  "pickup_address": "Pickup 1, Lagos",
  "dropoff_address": "Dropoff 1, Lagos",
  "pickup_latitude": 6.453000,
  "pickup_longitude": 3.391000,
  "dropoff_latitude": 6.501000,
  "dropoff_longitude": 3.351000,
  "estimated_amount_minor": 125000,
  "estimated_currency": "NGN",
  "driver_name_snapshot": "Driver 1",
  "vehicle_plate_snapshot": "LAG-101-AA",
  "started_at": "2026-10-06T18:07:13.612607Z",
  "completed_at": "2026-10-06T18:07:13.612607Z",
  "cancelled_at": null,
  "cancelled_by_role": null,
  "cancellation_fee_minor": null,
  "cancellation_fee_currency": null,
  "fare_amount_minor": 250000,
  "fare_currency": "NGN",
  "distance_m": 7100,
  "payments": [
    {"id": "832d20bb-9b1c-4c41-90a1-435dfc71f92b", "purpose": "fare", "status": "succeeded", "amount_minor":
250000, "currency": "NGN"}
  ],
  "created_at": "2026-10-06T18:07:13.612607Z",
  "updated_at": "2026-10-06T18:07:13.612607Z"
}
```


(The fare here differs from the quote because seed A wrote its rows directly; the contract says the fare equals the quote at completion. This does not affect the measurement.)

Size (CONFIRMED, M1; bytes are UTF-8 of the compact JSON, no spaces after : and ,; fields are top level keys plus 5 keys per payment):

Document	Fields	Bytes
EP03 rider view	31	971
EP03 driver view	21	698
EP03 support view	33	1,074
EP06 TripSummary item, rider list, before the contract fix	11	358
EP06 TripSummary item, rider list, after the contract fix	12	392

Header bytes (status line, headers, TLS) are not counted. Gzip would shrink repeated field names and ids; I did not measure it.

What each client uses

Client	What it shows	Fields it uses	Share of the response used	Share ignored
Rider trip list row	status, pickup, dropoff,	id, status,	from EP06 after the	EP06: 41.7% of fields,

	driver name, fare	pickup_address, dropoff_address, driver_name_snapshot, fare_amount_minor, fare_currency (7)	contract fix: 7 of 12 fields = 58.3%; 4,675 of 8,095 bytes on a page of 20 = 57.8%. (If it called EP03 per row instead: 7 of 31 fields = 22.6%, 221 of 971 bytes = 22.8%)	42.2% of bytes. EP03 per row: 77.4% of fields, 77.2% of bytes
Driver active trip card	where to pick up and drop off, the price	id, status, pickup_address, pickup_latitude, pickup_longitude, dropoff_address, dropoff_latitude, dropoff_longitude, estimated_amount_minor, estimated_currency (10)	10 of 21 fields = 47.6%; 309 of 698 bytes = 44.3%	52.4% of fields, 55.7% of bytes
Support console, trip page	everything about one trip	all 33 fields of EP03 plus history, payment detail and ratings	33 of 33 fields = 100% of EP03	0%, but it needs four requests: EP03 1,074 B + EP04 1,123 B + EP16 785 B + EP18 400 B = 3,382 B (CONFIRMED, M3)

Finding 1 (a contract gap, now closed in the contract; not built). The rider list row shows the driver's name, but the list item TripSummary of EP06 had no driver_name_snapshot. Closed in Stage 4 close: 06_api_contracts.md now gives the rider's TripSummary the field driver_name_snapshot. Measured on the real list page (rider 10000000-0000-0000-0000-000000000777, 20 rows, CONFIRMED, evidence/part_b_tripsummary_with_driver_name.txt): before the fix the EP06 page was 7,280 B (11 fields per item, 358 B for the seed item); after the fix it is 8,095 B (12 fields, 392 B for the seed item), that is 815 B more per 20 rows. The list row then uses 7 of the 12 fields, 4,675 of 8,095 bytes = 57.8% used, 42.2% ignored.

The GraphQL design sketch

The same need in GraphQL (design sketch, not run, nothing installed). Schema sketch for the types involved. Names are camel case as GraphQL convention; each maps to a column or a derived field.

```
type Trip {
  id: ID!
  status: TripStatus!
  pickup: Place!
  dropoff: Place              # null for a driver before accepting (field-level rule, see section 5)
  estimated: Money!           # estimated_amount_minor, estimated_currency
  fare: Money                 # fare_amount_minor, fare_currency
  cancellationFee: Money      # rider and support only
  driver: DriverSnapshot      # rider and support only
  distanceM: Int
  startedAt: DateTime  completedAt: DateTime  cancelledAt: DateTime
  payments: [Payment!]!       # rider and support only
  history: [StatusEvent!]!    # one resolver, reads trip_status_history ordered by seq
  ratings: [Rating!]!
  createdAt: DateTime!
}
```

The rider list row (status, pickup, dropoff, driver name, fare):

```
query RiderTripList($after: String) {
  myTrips(first: 20, after: $after, status: [COMPLETED, CANCELLED]) {
    items { id status pickup { address } dropoff { address } driver { name } fare { amountMinor currency } }
    nextCursor
  }
}
```

What it would return for the first item (values of the real seed trip 0001; the second to twentieth items have the same shape; the cursor is a placeholder):

```
{"data": {"myTrips": {"items": [
  {"id": "eeeeeeee-0000-0000-0000-000000000001", "status": "COMPLETED",
   "pickup": {"address": "Pickup 1, Lagos"}, "dropoff": {"address": "Dropoff 1, Lagos"},
   "driver": {"name": "Driver 1"}, "fare": {"amountMinor": 250000, "currency": "NGN"}}
 ], "nextCursor": "****"}}}
```

Size for the page of 20. The same seven fields as REST (the "needed" document) come to 4,675 B for the 20 rows of rider 777 (CONFIRMED; GraphQL's own envelope and nested keys such as pickup and fare would add some bytes that I did not compute). Against REST with the driver name in the summary (the contract after the fix), 8,095 B, that

is about 3.4 KB less per page (about 42% of the body). GraphQL's saving on this need is therefore real but small in absolute terms.

When REST is still fine

REST is the right start for this project. Three clients, all built by the same team, with the same release cycle. The over fetch measured is 42.2% of an 8,095 B list page of 20 rows (about 3.4 KB; CONFIRMED, evidence/part_b_tripsummary_with_driver_name.txt, line 17: 20 items, 8095 page bytes, 4675 used bytes, 42.2 percent ignored). Nobody notices that on a phone connection compared with TLS setup and the radio waking up. Every endpoint here has an exact contract with its ownership check, its errors by HTTP status (06b), its idempotency and its index (CONFIRMED plans in 06). That is easy to test, to rate limit and to cache. GraphQL would move all of that into resolvers. The expensive thing found is not the payload: finding 2 (a 14 to 17 ms sequential scan per trip read, before migration 016) would have been worse under GraphQL, because a resolver per row repeats it.

For comparison, the first 12 lines of the EP03 rider response (section 2):

```
{
  "id": "eeeeeeee-0000-0000-0000-000000000001",
  "status": "completed",
  "rider_id": "aaaaaaaa-0000-0000-0000-000000000001",
  "fare_quote_id": "ffffffff-0000-0000-0000-000000000001",
  "pickup_address": "Pickup 1, Lagos",
  "dropoff_address": "Dropoff 1, Lagos",
  "pickup_latitude": 6.453000,
  "pickup_longitude": 3.391000,
  "dropoff_latitude": 6.501000,
  "dropoff_longitude": 3.351000,
  "estimated_amount_minor": 125000,
```

How this API already limits over fetching (CONFIRMED sizes): an EP06 item is 392 B (after the driver name was added) against 971 B for the EP03 rider view (60% smaller); EP03 returns 698 B to a driver, 971 B to a rider and 1,074 B to support, because the contract hides fields by caller (data minimisation, B8); history (1,123 B for 5 events), ratings (400 B) and payment detail with events and refunds (785 B) are not in EP03; and an ETag on EP03 lets an unchanged trip be answered with a 304 and no 971 B body, though the server still runs the read (0.14 to 0.32 ms after migration 016, 13.8 to 14.3 ms before), so ETag saves bytes, not database time.

Cheap REST changes to make before considering GraphQL (none is built; decisions 2 to 4 below): driver_name_snapshot is in the rider's TripSummary (done in the contract, so the list row needs no second call, +815 B per 20 row page); a fields parameter on EP03 and the lists, with an allow list per role (for example ?fields=id,status,driver_name_snapshot), which gives the rider list row and the driver card the same 221 B and 309 B as the GraphQL queries; an expand parameter for the support console (?expand=history,payments,ratings), allowed for support only, which turns four requests into one; and a separate lightweight endpoint, only if a screen is hit very often. The payments lookup is served by payment_trip_id_idx (migration 016, decision 3).

When I would switch to GraphQL (any one is a reason to measure, two together are a reason to move): many client types with very different needs; a mobile bandwidth or latency problem measured in production; the number of one off endpoints or fields variants growing; screens that assemble data from many resources becoming the main way clients use the API, so that round trips, not bytes, dominate.

Decisions

- Stay on REST for the MVP. GraphQL is not needed for three internal clients; revisit on the switch points above.
- Add driver_name_snapshot to the rider's TripSummary (EP06). DONE in the contract in Stage 4 close (06_api_contracts.md); not built.
- Fix the payments by trip read. DONE: migration 016 added the ordinary index; the alternative not used was limiting EP03's payments to live payments. No migration was added in Part B; migration 016 was added in Stage 4 close.
- Optionally add fields and expand to EP03 and the lists when a client needs them, with the allow lists above. Not built.

The measuring queries for this chapter are db/queries/measure_over_fetch.sql, db/queries/measure_tripsummary_with_driver_name.sql and db/queries/part_b_payments_by_trip.sql.

Chapter 8. Real time

docs/08_real_time.md

The place and the choice

A3, the rider watching the trip. After a rider requests a trip, the rider app needs to know without asking again: the trip was accepted (assigned), the driver arrived, the trip started, it completed or was cancelled, and while the driver approaches, the driver's latest position. Two kinds of updates:

- Status changes. Rare (at most about 6 per trip), important, must never be lost or reordered. They are rows of trip_status_history, ordered by the seq column.
- Location pings. Frequent (the driver app sends one about every second through POST /v1/trips/{trip_id}/locations, EP13), only the latest matters, losing one is harmless.

Today the only way is polling EP03 (status) and EP05 (latest ping).

Comparing Server Sent Events and WebSockets

Question	Server Sent Events	WebSocket
Direction of traffic	One way: server to client. The rider only receives. The driver's phone sends pings by an ordinary POST (EP13), which the API already has and which carries authentication, ownership check, rate limit and idempotency rules per request.	Two ways on one connection. The rider app has nothing to send, so half of the channel is unused; and pings sent over a socket would need their own authentication and validation path.
Reconnection and resume	Built in for browsers: the client reconnects by itself and sends the Last Event ID header with the last id: it saw. The server resumes from there. Native app clients must implement the same rule (a few lines), INFERRED.	None built in. The application must invent its own resume message and its own reconnect timer.
Proxies and load balancers	It is a normal HTTP response that never ends, so it passes most proxies. It needs response buffering switched off (Cache Control: no store, X Accel Buffering: no) and a heartbeat shorter than the idle timeout of every hop (a comment line every 20 seconds). INFERRED.	Needs Upgrade support at every hop; some corporate proxies and older middleboxes break it. INFERRED.
HTTP versions and the browser connection limit	Over HTTP/1.1 a browser allows about 6 connections per host, and every open stream uses one, so many tabs of the same app can starve ordinary requests. A rider watches one trip, so it is one stream; over HTTP/2 streams share one connection. INFERRED.	Not held in the HTTP connection pool in the same way; its limits are different. INFERRED.
Mobile battery and background	A held connection keeps the radio awake. Both phone operating systems suspend network connections of an app in the background, so a stream only works while the app is in the foreground; "the driver arrived" in the background needs a push notification (outside this design). Sending a location event at most every 2 seconds, and only when it changed, limits radio use. INFERRED.	The same background rule applies. A socket needs its own keep alive ping to survive network changes. INFERRED.
Authentication of the stream	The stream is a GET, so it carries Authorization: Bearer ... like every endpoint (section 5, "Long lived token problem"). The browser EventSource object cannot set headers; a fetch based client or a library can. A token must never be put in the URL (it ends up in logs). INFERRED.	Browsers cannot set headers on the first WebSocket request either; the usual workarounds are a token in the URL or a first message. Both need more care than a header. INFERRED.
Scale	One open connection per watching rider; it holds a file descriptor and a small buffer on one API instance. I did not measure memory per connection. The number to plan with is open streams per instance, to be measured under load.	The same. A socket server also keeps per connection state for the application protocol.

What is sent	Status events from trip_status_history in seq order (the id: of each event is its seq), the latest ping as a location event, a heartbeat, and an end event.	The same messages, but the application defines the framing and acknowledgements.

The recommendation

Use SSE. The rider only receives, so a unidirectional channel is enough: SSE is plain HTTP, it resumes with a standard header, its authentication is the same as every other endpoint, and pings stay on the POST that already exists.

Switch to WebSockets when the client needs to send as well as receive on the same channel at a rate where a POST per message is a poor fit: in app chat between rider and driver, the driver app needing push commands from the server with acknowledgements, a live negotiation of the fare, or binary payloads. None of these is in the requirements.

Polling is the fallback, and it is what the apps can do today. Its cost, computed for N riders watching, each polling every 5 seconds (rate = N / 5 requests per second per endpoint). Re measured after migration 016 (the index on payment(trip_id)), CONFIRMED in evidence/migration_016_polling_remeasure.txt; the "before" column keeps the measurement of Part B and of the Stage 4 close baseline (evidence/migration_016_measure_before.txt).

Watching riders N	EP03 requests per second	EP03 body bytes per second (971 B each)	Database time per second, EP03 before the index (about 15 ms per read)	Database time per second, EP03 after (0.23 ms; range at 0.14 to 0.32 ms)	EP05 requests per second	Database time per second, EP05 (0.04 ms)
100	20	19.4 KB	0.30 s (0.3 core)	0.0046 s (0.003 to 0.006)	20	0.0008 s
1,000	200	194 KB	3.0 s (about 3 cores)	0.046 s (0.028 to 0.064)	200	0.008 s
10,000	2,000	1.9 MB	30 s (about 30 cores)	0.46 s (0.28 to 0.64)	2,000	0.08 s
100,000	20,000	19.4 MB	300 s (about 300 cores)	4.6 s (2.8 to 6.4)	20,000	0.8 s

Assumptions, visible:

- Poll interval 5 seconds, one EP03 read and one EP05 read per poll.
- Read times are warm cache execution times of the SQL statement (EXPLAIN ANALYZE), not whole request times: network, JSON building, connection pool, authentication and the application's own CPU are not included. EP03 read 0.23 ms (median of 5 runs, range 0.14 to 0.32 ms) after the index, against about 15 ms before (the payments scan inside it: 14.8 to 17.7 ms; the whole EP03 read: 13.8 to 14.3 ms; evidence/migration_016_measure_before.txt). EP05 read 0.04 ms (median of 4 warm runs; Part B measured 0.09 ms on another run). The first read of a trip that is not in cache cost 7 to 9 ms (EP03) and 3.8 ms (EP05); a rider who keeps polling keeps those pages warm, so the warm figure is used.
- Timings on this machine vary by about 3 times between runs, so every figure below is an order of magnitude, and the range column shows what the spread alone does.
- "Cores" = seconds of database execution per second of wall clock, assuming the work spreads perfectly over cores. Real systems lose some to locks and contention.

The feed and the LISTEN and NOTIFY tests

The API reads PostgreSQL; PostgreSQL does not push to the API unless the API asks. Proposal (one mechanism, in two steps):

- Step 1 (no change to the database): a batched poller inside each API instance. Each instance keeps the list of trips it has open streams for. Every 1 second it runs two queries for all of them at once: new status events from trip_status_history in seq order, and the latest ping per trip by a lateral join that takes the newest row of each trip. The cost grows with the number of open streams per instance, not with the number of polls: 10,000 streams over 10 instances is 1,000 streams per instance, one batch per second (INFERRED, scaling the 50 stream timing; not measured). Status events reach the rider within about 1 second.
- Step 2 (needs a migration in a later stage; NOT added now, decision 3): a doorbell. A NOTIFY sent when trip_status_history gets a row would wake the poller at once instead of waiting for its timer. The data path stays the same (read by seq), so the notification only needs to say "something changed for this trip".

What LISTEN and NOTIFY do and do not do (CONFIRMED, evidence/part_b_listen_notify.txt and part_b_listen_notify_cross_session.txt, run as ridehail_app with pg_notify by hand; no trigger, no table):

Test	Result	Meaning for the design
N2 notification sent in a transaction that is	not delivered	a rolled back status move never rings the

rolled back		bell; the notification is tied to the commit
N3c notification sent inside an open transaction	delivered only after the COMMIT	the reader never sees an event before the data it points to is visible
N4 notification sent while nobody listens, then a session starts listening	not delivered	notifications are lost while a listener is disconnected; they are not stored
N5 payload of 7,999 characters, then 8,000	the first is accepted; the second fails with SQLSTATE 22023 "payload string too long"	the payload limit is under 8,000 bytes: send ids only (trip_id, seq), never the event body

Why seq makes resume safe. Notifications can be lost (N4), but they are never the data. Whenever the API wakes (by a notification or by its 1 second timer) it reads from the table WHERE trip_id = ... AND seq > last_sent. A lost notification costs at most one timer interval of delay, never an event. The same read serves a client that reconnects with Last Event ID.

The event format, resume and ending

The endpoint is EP27 GET /v1/trips/{trip_id}/events, specified in 06_api_contracts.md. Response: 200, Content Type: text/event stream, Cache Control: no store, X Accel Buffering: no.

A first connection (real seq values and statuses of trip 40000000-0000-0000-0000-000000198904; the timestamps are identical because the bulk loader wrote the whole trip in one transaction; the location line is illustrative):

```
GET /v1/trips/40000000-0000-0000-0000-000000198904/events
Authorization: Bearer ****
Accept: text/event-stream

HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-store

retry: 3000

id: 930297
event: status
data: {"seq":930297,"from_status":null,"to_status":"requested","created_at":"2026-10-06T18:43:32.458283Z"}

id: 930298
event: status
data: {"seq":930298,"from_status":"requested","to_status":"assigned","created_at":"2026-10-06T18:43:32.458283Z"}

event: location
data: {"latitude":6.455210,"longitude":3.388742,"accuracy_m":8,"recorded_at":"2026-10-06T18:44:09.310552Z"}

: keepalive
```

Resume with seq. The client had seen 930299 (arrived) and lost the connection. It reconnects with that id and receives only what it missed, then the end:

```
GET /v1/trips/40000000-0000-0000-0000-000000198904/events
Last-Event-ID: 930299

HTTP/1.1 200 OK
...
id: 930300
event: status
data: {"seq":930300,"from_status":"arrived","to_status":"started","created_at":"2026-10-06T18:43:32.458283Z"}

id: 930301
event: status
data: {"seq":930301,"from_status":"started","to_status":"completed","created_at":"2026-10-06T18:43:32.458283Z"}

event: end
data: {"reason":"completed"}
```

The test outputs are evidence/part_b_listen_notify.txt, evidence/part_b_listen_notify_cross_session.txt, evidence/part_b_realtime_plans.txt, evidence/part_b_trip_row_lock_b.txt, evidence/migration_016_polling_remeasure.txt.

Chapter 9. The proof: plans and the three rejected inserts

Built from docs/04_hard_questions.md (question 7), evidence/plans_for_grader_after_016.txt, evidence/proofs_rejected_and_extra.txt, evidence/migration_016_measure_before.txt and evidence/migration_016_measure_after.txt.

The first choice of index was made by reasoning (INFERRED). The choices were later measured with EXPLAIN (ANALYZE) on the built database (CONFIRMED). For the brief, the two heaviest queries are A2.1 (the heaviest read, and the only one whose cost grows with data it does not return) and the A1.3 trip insert (the heaviest statement of all, 7 to 31 ms depending on cache state, almost all of it in the two triggers trip_enforce_transition and trip_write_history). The second heaviest read is under a millisecond and uses a single index lookup, so it shows nothing the other plans do not.

The two heaviest queries

A2.1 is the driver's app asking for open requests near its position. The query is served in part by the index trip_requested_created_at_idx ON trip (created_at) WHERE status = 'requested'. A partial index holds only waiting trips, and it stays small only after VACUUM: every trip that leaves requested leaves a dead entry in it until vacuum removes it. The A2.1 query does not read it in order and stop after 20 rows. The rectangle is on fare_quote, so Postgres must first find all waiting trips, then look up each trip's quote, filter by the rectangle, sort and cut to 20. With 401 waiting trips that is a bitmap scan, a sort and 401 lookups. It will not scale with a large waiting list: the cost grows with the number of waiting trips, not with the 20 rows returned.

Measured on the real database (Stage 3, bulk set of 200,012 trips, 401 waiting, EXPLAIN (ANALYZE, BUFFERS), as ridehail_app; CONFIRMED). Right after the bulk load the query was 20.6 ms, 5,817 buffers. There was a Bitmap Heap Scan on trip (3,956 heap blocks) over a BitmapAnd of trip_one_active_per_driver (24,446 entries) and trip_requested_created_at_idx (9,354 entries) for only 401 real rows, then 401 Index Scan using fare_quote_pkey lookups, then Sort and Limit. No Seq Scan. The extra entries were dead index entries from 200,000 trips that had left requested. After VACUUM (ANALYZE) trip, payment the query was 3.4 ms, 1,713 buffers, entries 708 and 403, 45 heap blocks, with the same plan shape.

The planner also used trip_one_active_per_driver (a partial index on the four active statuses) in the BitmapAnd. It is valid because status = 'requested' implies the four status condition, but it is not the index the doc planned for, and it adds work. The 401 fare_quote lookups are the cost that remains, so the plan is a bitmap scan plus a sort plus 401 lookups, not a short ordered scan.

```
 Limit  (cost=5169.00..5169.03 rows=12 width=64) (actual time=2.093..2.102 rows=20 loops=1)
   ->  Sort  (cost=5169.00..5169.03 rows=12 width=64) (actual time=2.091..2.097 rows=20 loops=1)
         ->  Nested Loop  (cost=2151.06..5168.78 rows=12 width=64) (actual time=1.341..2.030 rows=22 loops=1)
               ->  Bitmap Heap Scan on trip t  (cost=2150.64..2154.65 rows=407 width=58) (actual
time=0.291..0.557 rows=401 loops=1)
                     Recheck Cond: ((status = ANY ('{requested,assigned,arrived,started}'::text[])) AND (status
= 'requested'::text))
                     ->  BitmapAnd  (cost=2150.64..2150.64 rows=1 width=0) (actual time=0.273..0.274 rows=0
loops=1)
                           ->  Bitmap Index Scan on trip_one_active_per_driver  (cost=0.00..811.88 rows=720
width=0) (actual time=0.158..0.158 rows=761 loops=1)
                           ->  Bitmap Index Scan on trip_requested_created_at_idx  (cost=0.00..1338.31 rows=407
width=0) (actual time=0.103..0.103 rows=453 loops=1)
               ->  Index Scan using fare_quote_pkey on fare_quote q  (cost=0.42..7.41 rows=1 width=38) (actual
time=0.003..0.003 rows=0 loops=401)
                     Index Cond: (id = t.fare_quote_id)
                     Filter: ((pickup_latitude >= 6.475) AND (pickup_latitude <= 6.495) AND (pickup_longitude >=
3.385) AND (pickup_longitude <= 3.405))
                     Rows Removed by Filter: 1
```

Re measured (follow up 2, after VACUUM (ANALYZE) of trip, payment and trip_status_history; CONFIRMED). A2.1: 3.46 ms, 1,722 buffers, same plan shape: Bitmap Heap Scan on trip (54 heap blocks) over a BitmapAnd of trip_one_active_per_driver (726 entries) and trip_requested_created_at_idx (420 entries), then 401 Index Scan using fare_quote_pkey, Sort, Limit. No Seq Scan. After migration 013 the same query measured 1.1 to 1.5 ms on two further runs, so single timings on this machine vary by a factor of about 3. The conclusion does not change: the cost is the 401 quote lookups and grows with the waiting list.

A1.3 is the trip insert, the heaviest WRITE, and it includes the trigger work. It was rolled back. The plan shows an Insert on trip with a Result node, and then the trigger times. A plan does not list the indexes the insert has to maintain; the indexes of trip are trip_driver_history_idx, trip_fare_quote_id_key, trip_one_active_per_driver, trip_one_active_per_rider, trip_pkey, trip_requested_created_at_idx and trip_rider_history_idx. The insert is the heaviest statement of all, almost all of its time in the two triggers trip_enforce_transition and trip_write_history, against the four foreign key triggers that are each well under a millisecond.

```
 Insert on trip  (cost=0.00..0.02 rows=0 width=0) (actual time=3.886..3.887 rows=0 loops=1)
   Buffers: shared hit=161 read=1 dirtied=1
   ->  Result  (cost=0.00..0.02 rows=1 width=364) (actual time=0.096..0.097 rows=1 loops=1)
```

```
 Planning Time: 0.100 ms
 Trigger for constraint trip_rider_id_fkey: time=0.180 calls=1
 Trigger for constraint trip_driver_id_fkey: time=0.069 calls=1
 Trigger for constraint trip_vehicle_belongs_to_driver: time=0.034 calls=1
 Trigger for constraint trip_quote_belongs_to_rider: time=0.253 calls=1
 Trigger trip_enforce_transition: time=2.292 calls=1
 Trigger trip_write_history: time=3.337 calls=1
 Execution Time: 7.875 ms
```

The three rejected inserts

The three rejected inserts below, plus the two variants, each show a rule refusing a write on the built database, rolled back. The error messages are copied exactly.

Rule	Statement in words	Error message	Constraint or trigger that refused it
B1	rider 5 already has an active trip (trip 09, requested) and requests another	duplicate key value violates unique constraint "trip_one_active_per_rider"	partial unique index trip_one_active_per_rider
B7	rating on a trip that is not completed (trip 12 is started), by its rider	a rating needs a completed trip (status is started) (B7)	trigger rating_enforce_rules
B7	a second rating by the same side (rider 1 already rated trip 01)	duplicate key value violates unique constraint "rating_one_per_role_per_trip "	unique constraint rating_one_per_role_per_trip
B3	payment with currency USD for trip 01 (rider 1, correct amount 250000)	new row for relation "payment" violates check constraint "payment_currency_ngn"	check constraint payment_currency_ngn
B3	payment with a negative amount (-5) for trip 01	payment amount -5 does not equal the trip fare 250000	trigger payment_enforce_rules

Stage 3 rule: demonstrations of rejected inserts must run as the role ridehail_app. Each demonstration of a refused insert or update (a second active trip, a negative amount, an edit of a completed trip, and so on) must be run after SET ROLE ridehail_app (or SET LOCAL ROLE inside a rolled back transaction), never as the dev login ridehail. Reasons: (1) ridehail is a superuser (CONFIRMED, rolsuper = t). A superuser skips privilege checks, so a "permission denied" can never appear, and a missing grant would stay invisible. (2) A superuser can switch triggers off or change tables, so a demonstration run that way does not show what the real application can do. (3) Several of my rules depend on current_user (the erasure rule). Every proof ran after SET ROLE ridehail_app from the dev login and printed current_user.

Rejected insert 1 (B1): a second active trip for rider 5, refused as ridehail_app.

```
 current_user | session_user
--------------+--------------
 ridehail_app | ridehail
ERROR:  duplicate key value violates unique constraint "trip_one_active_per_rider"
```

```
--- 1. B1: rider 5 already has an active trip (trip 09, requested) and requests another
 current_user | session_user
--------------+--------------
 ridehail_app | ridehail
(1 row)

 set_config |              set_config
------------+--------------------------------------
 rider      | aaaaaaaa-0000-0000-0000-000000000005
(1 row)

ERROR:  duplicate key value violates unique constraint "trip_one_active_per_rider"
DETAIL:  Key (rider_id)=(aaaaaaaa-0000-0000-0000-000000000005) already exists.
--- 2a. B7: rating on a trip that is not completed (trip 12 is started), by its rider
 current_user | session_user
--------------+--------------
 ridehail_app | ridehail
```

Image 9.1. A second active trip for rider 5 is refused by trip_one_active_per_rider, as ridehail_app.

Rejected insert 2a (B7): a rating on an unfinished trip (trip 12 started), refused as ridehail_app.

```
 current_user | session_user
--------------+--------------
 ridehail_app | ridehail
ERROR:  a rating needs a completed trip (status is started) (B7)
```

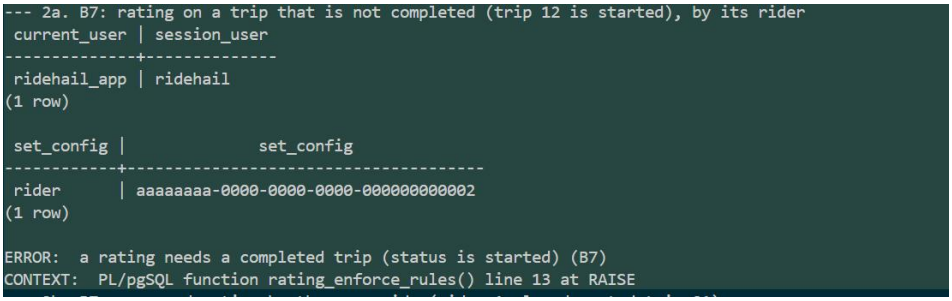


Image 9.2. A rating on an unfinished trip (trip 12 started) is refused by rating_enforce_rules, as ridehail_app.

Rejected insert 2b (B7): a second rating by the same side, refused as ridehail_app.

```
 current_user | session_user
--------------+--------------
 ridehail_app | ridehail
ERROR:  duplicate key value violates unique constraint "rating_one_per_role_per_trip"
```

Rejected insert 3a (B3): a payment in USD for trip 01, refused as ridehail_app.

```
 current_user | session_user
--------------+--------------
 ridehail_app | ridehail
ERROR:  new row for relation "payment" violates check constraint "payment_currency_ngn"
```

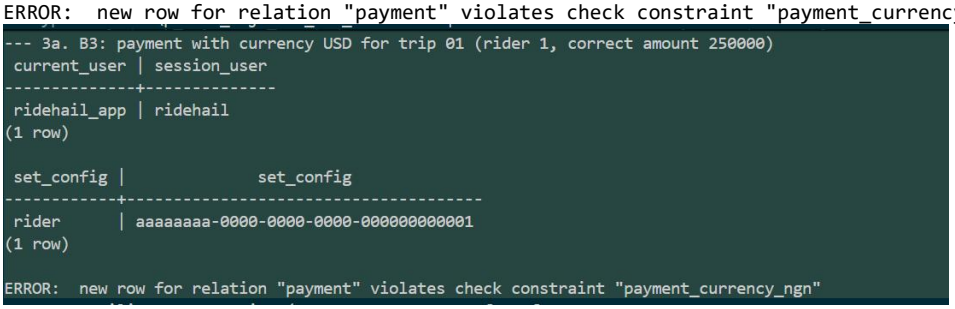


Image 9.3. A payment in USD for trip 01 is refused by payment_currency_ngn, as ridehail_app.

Rejected insert 3b (B3): a payment with a negative amount (-5) for trip 01, refused as ridehail_app.

```
 current_user | session_user
--------------+--------------
 ridehail_app | ridehail
ERROR:  payment amount -5 does not equal the trip fare 250000
```

The limits

Timings on this machine vary by about 3 times between runs, so every millisecond figure in these docs is an order of magnitude, not a promise. The plans themselves are orders of magnitude apart where an index was added: before migration 016 a read of a trip's payments was a parallel Seq Scan of the whole payment table (169,410 rows, 14.5 to 17.7 ms), and after the migration the same read is an Index Scan using payment_trip_id_idx (0.06 to 0.10 ms), while the EP03 full read fell from 13.8 to 16.2 ms to 0.14 to 0.32 ms (CONFIRMED, evidence/migration_016_measure_before.txt and evidence/migration_016_measure_after.txt).

All other proofs (the error mapping, the 45 SQLSTATE tests, the search path attacks, the login role test and the idempotency concurrency test) are listed by evidence file name in Appendix C.

Chapter 10. Open items and decisions

Built from docs/09_open_items.md and docs/10_brief_checklist.md.

Decisions recorded

Recorded on 7 October 2026. This section records what the owner decided. Where section 2 above still reads earlier than this record, this record is the later word. Twelve decisions are approved and five are still open.

Approved

- D10, erasure of text columns. Approved: the five text columns (pickup_address, dropoff_address, driver_name_snapshot and vehicle_plate_snapshot on trip, and comment on rating) may be blanked only by erase_rider and erase_driver, running under ridehail_erasure_owner.
- D1, fare model. Approved as written.
- D2, cancellation fee. Approved as written.
- D3, request timeout and payment reconcile after. Approved as written.
- D4, settings values. Approved as written.
- D6, rate limits. Approved as written.
- D7, support visibility. Approved as written.
- D11, the 62 bulk trips with a misplaced seq. Approved as written.
- D12, manual refunds of failed payments leave no money record. Approved as written.
- D14, real time with SSE. Approved as written.
- D15, REST fields and expand parameters, and staying on REST. Approved as written.
- D17, the driver name in the rider's TripSummary. Approved as written.

Still open

- D5, webhook provider. Design assumption written in docs\06 section 1.11 and in EP15, EP19 and EP21; owner may replace.
- D8, Q5 location retention.
- D9, Q6 a driver who never arrives.
- D13, authentication design. Design assumption written in docs\06 section 1.10 (baseline: short lived tokens with a person status check on every request; a denylist is an optional hardening); owner may replace.
- D16, distance estimator. Design assumption written in docs\06 EP01 (the price rule and the distance estimator interface); owner may replace.

Placeholders

Every placeholder is a value I chose so that something can run; it needs the owner.

#	Placeholder	Current value	Where it lives	Why it is a placeholder
P1	City	Lagos (assumed)	db/seed/seed_sample_settings.sql, applied; used by the service area box	no city was ever given
P2	Base fare	50000 kobo (500 naira)	setting base_fare_minor	no fare model was given
P3	Price per kilometre	15000 kobo (150 naira)	setting per_km_minor	same
P4	Price per minute	3000 kobo (30 naira)	setting per_minute_minor	same
P5	Quote validity	300 seconds	setting quote_validity_seconds	same
P6	Service area box	6.38 to 6.70 north, 3.10 to 3.70 east (stored as degrees x 1,000,000: 6380000, 6700000, 3100000, 3700000)	settings service_area_min_lat_e6, service_area_max_lat_e6, service_area_min_lon_e6, service_area_max_lon_e6	a rough rectangle; it also contains sea and land the service will not cover
P7	Timeout job interval	60 seconds	setting request_timeout_check_interval_seconds	not chosen by the owner
P8	Payment reconciliation job interval	300 seconds	setting payment_reconcile_check_interval_seconds	not chosen by the owner
P9	Request timeout	600 seconds	setting request_timeout_seconds (B9, T9)	marked "placeholder, needs owner approval" in 01, 03, 05
P10	Payment reconcile after	900 seconds	setting payment_reconcile_after_seconds	marked "placeholder, needs owner approval" in 03, 05
P11	Cancellation fee	50000 kobo (500 naira), flat, after assignment	setting cancellation_fee_minor (B9)	the owner approved it as a placeholder held in a setting, not hard coded
P12	Test support admin	dddddddd-0000-0000-0000-000000000001	named in the settings seed; the 10 audit rows of the seed credit it	a seed A test row; a real support admin must replace it before real use

P13	Distance and duration estimator	straight line distance x 1.3, speed 8.33 m/s	06 EP01	the routing method is not designed
P14	Rate limits	quotes 10/min, pings 1/s per trip, other writes 30/min, reads 120/min	06 section 1.8	chosen without traffic figures
P15	Webhook scheme	HMAC SHA256 of timestamp + body, 5 minute window, fields provider_event_id, provider_reference, event_type, amount_minor, currency, occurred_at, failure_reason	06 EP19	the provider is unnamed
P16	Idempotency timings	key lasts 24 hours; a stuck payment key is taken over after 120 seconds; lock wait 800 ms	06 section 1.7, EP15	chosen, not measured against real traffic
P17	API limits	addresses 1 to 300 characters, reasons 1 to 500, notes 1 to 2000, comment up to 500, body up to 16 KiB, list page 20 (maximum 100), open requests maximum 50	06	chosen
P18	Event stream settings	heartbeat 20 s, location one event per 2 s, 15 minute maximum life, close at token expiry, 2 open streams per rider per trip, poller every 1 s	06 EP27, 08	designs; nothing was served over HTTP
P19	Cursor secret	not created	06 section 1.9	the signing secret belongs in the environment, never in the project

Known limits

What the design does not do, or has not proved.

- 62 bulk trips (299 history rows) have a history row numbered out of order (05 item 13, CONFIRMED in evidence/migration_010_seq_investigation.txt). Accepted by the owner. A repair would need the append only trigger switched off, which nobody approved.
- pg_hba trusts loopback and the local socket inside the container. Any process in the container connects as any role with no password. Only other addresses use scram sha-256 (CONFIRMED in the login role test). Not changed (05 item 20).
- Only three table names were attacked in the search path proofs (audit_log, trip_status_history, status_transition_rule). The other tables rely on the same two protections (migrations 013 and 014) but were not each attacked (05 item 21).
- Almost no concurrency tests exist beyond two: a second insert of the same idempotency key waits and times out (55P03), and a second move of the same trip waits for the first. Nothing was raced for two drivers accepting the same trip, two refunds on one payment, or a repeated provider event (05 item 10).
- No driver is installed, so the SQLSTATE field name is unconfirmed in any driver. 06 section 1.6 gives the names I expect for node postgres, pgx, psycopg and JDBC from memory. Nothing was installed.
- Nothing was run over HTTP: no API exists (the api\ folder is empty), no endpoint, no stream, no GraphQL, no webhook. The contracts were checked only as the SQL behind them (06, 07, 08).
- The actor is claimed by the application. The database checks that the claimed actor may make a move; it cannot prove who the actor is. A login role that is a member of ridehail_app can claim support (CONFIRMED, 05 items 7 and 18). A superuser can still switch triggers off.
- Q5 and Q6 are open. Q5, location retention, has no period chosen and is blocked on legal advice (NDPA 2023 was not read). Q6, a driver who never arrives, still lets the rider pay the fee if they cancel after assignment. Both are open owner decisions (D8 and D9).

The document itself shows only the four unfilled class note places, in chapters 7 and 8.

Appendix

Built from docs/11_word_outline.md (Appendix, lines 124 to 128) and evidence/00_index.md (section B and the later sections), with the real folder listings of docs, db/migrations, db/queries and db/seed.

This appendix gathers the long lists of names that would otherwise run to pages of text: the documents, the migrations, the evidence, the queries and the seeds. Every name below is a real file name read from the named folders and the evidence index.

Appendix A, documents

The documents, from docs/: docs/01_requirements.md, docs/02_entities.md, docs/02b_relationships.md, docs/03_state_machine.md, docs/04_hard_questions.md, docs/05_traceability.md, docs/06_api_contracts.md, docs/06b_error_catalogue.md, docs/06c_endpoint_matrix.md, docs/07_over_fetching.md, docs/08_real_time.md, docs/09_open_items.md, docs/10_brief_checklist.md, docs/11_word_outline.md, docs/12_brief_coverage.md, docs/13_defence_answers.md, docs/14_post.md, docs/15_appendix_f_entities.md.

Appendix B, migrations

The sixteen migrations, by file name, from db/migrations/: db/migrations/001_roles_and_functions.sql, db/migrations/002_reference_tables.sql, db/migrations/003_people.sql, db/migrations/004_quotes_trips.sql, db/migrations/005_money.sql, db/migrations/006_rating_idempotency_ping_erasure.sql, db/migrations/007_indexes.sql, db/migrations/008_grants.sql, db/migrations/009_fix_erasure_select_grants.sql, db/migrations/010_history_audit_written_by_database.sql, db/migrations/011_fix_add_support_note.sql, db/migrations/012_drop_old_history_index_revoke_public_execute.sql, db/migrations/013_search_path_pg_temp_last.sql, db/migrations/014_revoke_temp_from_public.sql, db/migrations/015_sqlstate_per_rule.sql, db/migrations/016_payment_trip_id_index.sql.

Appendix C, evidence

The evidence files, by the groups of evidence/00_index.md section B and the later sections, by group and file name only.

- Migrations: evidence/migration_001_applied.txt, evidence/migration_002_applied.txt, evidence/migration_003_applied.txt, evidence/migration_004_applied.txt, evidence/migration_005_applied.txt, evidence/migration_006_applied.txt, evidence/migration_007_008_applied.txt, evidence/migration_007_008_show.txt, evidence/migration_009_applied.txt, evidence/migration_010_before.txt, evidence/migration_010_dry_run.txt, evidence/migration_010_applied.txt, evidence/migration_010_seq_investigation.txt, evidence/migration_011_applied.txt, evidence/migration_012_applied.txt, evidence/migration_013_applied.txt, evidence/migration_014_applied.txt
- Catalogue checks: evidence/catalogue_actual.txt, evidence/catalogue_check.txt, evidence/catalogue_check_after_009.txt, evidence/catalogue_and_passes_after_010.txt, evidence/followup2_catalogue_and_passes.txt, evidence/followup3_catalogue_and_passes.txt
- Seeds and the bulk load: evidence/seed_a_applied.txt, evidence/seed_a_counts.txt, evidence/seed_b_timing_first_chunk.txt, evidence/seed_b_parallel_log_run1_failed_integer_overflow.txt, evidence/seed_b_parallel_err_run1_0.txt, evidence/seed_b_parallel_err_run1_1.txt, evidence/seed_b_parallel_err_run1_2.txt, evidence/seed_b_parallel_err_run1_3.txt, evidence/seed_b_parallel_log.txt, evidence/seed_b_parallel_err_0.txt, evidence/seed_b_parallel_err_1.txt, evidence/seed_b_parallel_err_2.txt, evidence/seed_b_parallel_err_3.txt, evidence/seed_b_final_counts.txt, evidence/vacuum_after_bulk.txt
- The five queries and the plans: evidence/query_parameters.txt, evidence/queries_a1_a5_results.txt, evidence/plans_all_candidates_warm.txt, evidence/plan_heaviest_1_A2_open_requests.txt, evidence/plan_heaviest_1_A2_after_vacuum.txt, evidence/plan_heaviest_2_A1_insert_trip.txt, evidence/plans_all_candidates_after_vacuum.txt, evidence/plans_partial_and_unused_indexes.txt, evidence/followup2_vacuum.txt, evidence/followup2_plans_all_candidates.txt, evidence/followup2_plans_selected_reads.txt, evidence/followup2_plans_after_013.txt, evidence/followup2_plans_after_013_run3.txt
- Proofs of rules (rejected inserts, extras, history and audit): evidence/proofs_rejected_and_extra.txt, evidence/proofs_run1_erasure_failed_before_migration_009.txt, evidence/proofs_migration_010.txt, evidence/proofs_migration_010_run1_before_migration_011.txt, evidence/followup2_proofs_a_c.txt
- Search path, login role, index drop, TEMP revoke: evidence/followup2_groundwork.txt, evidence/followup2_proof_1b_before_fix.txt, evidence/followup2_proof_1b_E2_clean_before_fix.txt, evidence/followup2_proof_1b_after_fix.txt, evidence/followup2_013_dry_run.txt, evidence/followup2_login_role_test.txt, evidence/followup2_012_a_index_usage.txt, evidence/followup2_012_a_which_statements.txt, evidence/followup2_012_dry_run.txt,

evidence/followup3_temp_groundwork.txt, evidence/followup3_014_dry_run.txt, evidence/followup3_014_proofs.txt, evidence/followup3_attacks_rerun_after_014.txt

- State of the project: evidence/stage3_followup_final_state.txt
- Stage 4 follow up, migration 015 (one SQLSTATE per rule): evidence/migration_015_catalogue_before.txt, evidence/migration_015_catalogue_after.txt, evidence/migration_015_catalogue_diff.txt, evidence/migration_015_dry_run.txt, evidence/migration_015_applied.txt, evidence/migration_015_rerun_api_error_mapping.txt, evidence/migration_015_error_tests_compare.txt, evidence/migration_015_rerun_walkthrough.txt, evidence/migration_015_rerun_rejected_and_extra.txt, evidence/migration_015_rerun_proofs_migration_010.txt, evidence/migration_015_rerun_attacks_search_path.txt, evidence/migration_015_rerun_temp_revoke.txt, evidence/migration_015_rerun_followup2_a_c.txt, evidence/migration_015_rerun_list_endpoints.txt, evidence/migration_015_rerun_api_walkthrough.txt, evidence/migration_015_state_after.txt
- Stage 4 Part A close (20 untested codes, settings seed): evidence/untested_rh_codes_proofs.txt, evidence/seed_sample_settings_before.txt, evidence/seed_sample_settings_applied.txt, evidence/seed_sample_settings_ep01_after.txt, evidence/seed_sample_settings_dry_run.txt
- Stage 4 Part B (over fetching and real time): evidence/part_b_over_fetch_measurements.txt, evidence/part_b_payments_by_trip.txt, evidence/part_b_realtime_plans.txt, evidence/part_b_listen_notify.txt, evidence/part_b_listen_notify_cross_session.txt, evidence/part_b_listen_notify_cross_session.sender.txt, evidence/part_b_trip_row_lock_a.txt, evidence/part_b_trip_row_lock_b.txt
- Stage 4 close (migration 016, contract fix, re measure): evidence/migration_016_measure_before.txt, evidence/migration_016_measure_after.txt, evidence/migration_016_dry_run.txt, evidence/migration_016_applied.txt, evidence/migration_016_insert_cost_paired.txt, evidence/migration_016_polling_remeasure.txt, evidence/part_b_tripsummary_with_driver_name.txt, evidence/catalogue_check_016.txt
- Files with no earlier entry (added in fix 21): evidence/api_error_mapping_tests.txt, evidence/api_walkthrough_tests.txt, evidence/api_list_endpoint_tests.txt, evidence/api_idempotency_concurrency_test.txt, evidence/api_idempotency_concurrency_session_a.txt, evidence/catalogue_check_stage5_prep.txt, evidence/catalogue_check_stage5_prep2.txt, evidence/catalogue_check_stage5_prep3.txt, evidence/plans_for_grader_after_016.txt, evidence/stage4_close_state_after.txt, evidence/stage5_export2_composite_fk.txt, evidence/stage5_export4_f3_race.txt, evidence/stage5_prep2_design_columns.txt, evidence/stage5_prep2_fare_arithmetic.txt

The full index, with what each file shows, is in evidence/00_index.md.

Appendix D, queries and seeds

The queries, by file name, from db/queries/: db/queries/a1_a5_queries.sql, db/queries/catalogue_015.sql, db/queries/check_design_assumption_columns.sql, db/queries/check_docs_vs_db.ps1, db/queries/check_fare_arithmetic.sql, db/queries/dry_run_015.sql, db/queries/dry_run_016.sql, db/queries/measure_over_fetch.sql, db/queries/measure_tripsummary_with_driver_name.sql, db/queries/migration_016_insert_cost_paired.sql, db/queries/migration_016_measure.sql, db/queries/migration_016_polling_remeasure.sql, db/queries/part_b_listen_notify.sql, db/queries/part_b_listen_notify_cross_session.sql, db/queries/part_b_payments_by_trip.sql, db/queries/part_b_realtime_plans.sql, db/queries/part_b_trip_row_lock_a.sql, db/queries/part_b_trip_row_lock_b.sql, db/queries/plans_all_candidates.sql, db/queries/plans_for_grader.sql, db/queries/plans_partial_and_unused_indexes.sql, db/queries/proofs_api_error_mapping.sql, db/queries/proofs_api_list_endpoints.sql, db/queries/proofs_api_walkthrough.sql, db/queries/proofs_ep01_after_seed.sql, db/queries/proofs_followup2_012_checks.sql, db/queries/proofs_followup2_a_c.sql, db/queries/proofs_followup2_b_search_path.sql, db/queries/proofs_followup2_login_role.sql, db/queries/proofs_followup3_temp_revoke.sql, db/queries/proofs_migration_010.sql, db/queries/proofs_rejected_and_extra.sql, db/queries/proofs_untested_rh_codes.sql. The seeds, by file name, from db/seed/: db/seed/seed_a_small.sql, db/seed/seed_b_0_people_quotes.sql, db/seed/seed_b_1_trips_chunk.sql, db/seed/seed_sample_settings.sql, db/seed/seed_sample_settings.before_fix11.sql.

Appendix E, full text of two endpoints

This appendix holds the full text of two endpoints, EP02 and EP15, copied from docs/06_api_contracts.md. The other endpoints stay in that file. EP01, EP12 and EP27 are referenced by file name at the end of this section.

EP02 POST /v1/trips (rider app; A1, B0, B1, B6)
Who and ownership: role rider. The trip's rider_id is sub. The quote must satisfy fare_quote.id = body.fare_quote_id AND fare_quote.rider_id = sub.
Request: headers Authorization, Content Type, Idempotency Key (required). Body:

Field	Type	Required	Limits
fare_quote_id	uuid	yes	a quote of this rider that has not expired and has not started a trip
pickup_address	string	yes	1 to 300 characters, not blank
dropoff_address	string	yes	1 to 300 characters, not blank

Flow (one transaction): set the actor; insert the idempotency row; read the quote with the ownership condition; INSERT INTO trip (id, rider_id, fare_quote_id, pickup_address, dropoff_address) (status defaults to requested; the trigger writes the history row (new) > requested); UPDATE idempotency_key SET response_status = 201, response_body = ..., trip_id = ... CONFIRMED in the walkthrough, evidence/api_walkthrough_tests.txt.
Success: 201, header Location: /v1/trips/{id}, body Trip (rider view, status = requested). A replay returns the same 201 body with Idempotent Replay: true.
Errors:

Status	Code	When	Database cause
409	rider_has_active_trip	the rider already has a trip in requested, assigned, arrived or started	unique index trip_one_active_per_rider, "duplicate key value violates unique constraint "trip_one_active_per_rider"" (E01)
404	quote_not_found	no quote with that id for this rider (unknown id, or another rider's quote)	ownership query; backstop FK trip_quote_belongs_to_rider, "insert or update on table "trip" violates foreign key constraint "trip_quote_belongs_to_rider"" (E02a, E02b)
409	quote_already_used	the quote already started a trip	unique trip_fare_quote_id_key (E03)
422	quote_expired	the quote's expires_at has passed	trigger trip_enforce_transition, SQLSTATE RH004 (message "fare quote % has expired", E04); the API also checks expires_at first
400	invalid_idempotency_key	header missing, empty, longer than 128, or has a space	backstop check idem_key_length (E32)
422	idempotency_key_reuse	the key was used with a different body	none: the stored request_hash differs
409	request_in_progress	the same key is still being processed	SQLSTATE 55P03 on the idempotency insert (concurrency test)
422	validation_failed	an address is blank or too long, fare_quote_id is not a uuid, a field is unknown	none

Also: 401, 403, 429, 500, 503 (common). Order of checks: replay and key checks come first, so a retry with the same key never returns rider_has_active_trip (it replays the 201). A new key after a success does return it.
Idempotency: keyed (section 1.7a), action request_trip. A retry cannot create a second trip: the stored answer is replayed, and even without it trip_one_active_per_rider and trip_fare_quote_id_key refuse a second trip.
Session settings: rider, sub.

EP15 POST /v1/trips/{trip_id}/payments (rider app; A4, B3, B6, U1)
Who and ownership: role rider; trip.rider_id = sub.
Request: headers Authorization, Content Type, Idempotency Key (required). Body: payment_method_token (string, 1 to 200 characters, an opaque token from the provider's own client library; the API never receives or stores card data). The client sends no amount and no purpose. The API reads the trip: status completed gives purpose = fare and amount_minor = fare_amount_minor; status cancelled with cancellation_fee_minor > 0 gives purpose = cancellation_fee and that fee. Currency is NGN.
Flow (three transactions, because a provider call must not hold a database lock):
1. Transaction 1: set the actor; INSERT INTO idempotency_key ... ON CONFLICT DO NOTHING (action pay_trip, request_hash includes the trip_id); commit. Replay rules of 1.7a apply here.
2. Call the provider to start the charge, passing our Idempotency Key as the provider's own idempotency key (assumption, decision 5). The provider returns a provider_reference. Provider unreachable or refusing: delete the key row, 502 provider_error, no payment row exists.

3. Transaction 2: set the actor; INSERT INTO payment (id, trip_id, purpose, amount_minor, currency, provider_reference) (status pending); UPDATE idempotency_key SET response_status = 202, response_body = ..., payment_id = ...; commit.
If the process dies after step 2, a retry after 120 seconds repeats step 2 (the provider returns the same reference for the same key) and finishes step 3. If the client never retries, the provider's later webhook finds no payment with that reference and is stored as needs_support (EP19). This is a known gap.
Provider capabilities used here (D5, DESIGN ASSUMPTION, owner may replace; full list in section 1.11). Step 2 needs capability C1: create a payment with our idempotency key and get back a provider reference, with the same key giving the same reference and no second charge. Step 3 needs the reference to be unique (payment_provider_reference_key). The recovery after a crash (the retry after 120 seconds) needs C1 to behave that way. If a provider lacks C1, the retry is unsafe and the design must change: a payment attempt has to be recorded before the provider is called, which needs a migration, because payment.provider_reference is required today. Testing this endpoint needs C5, a sandbox.
Success: 202, header Location: /v1/payments/{id}, body {"payment": Payment} with status = pending. The result arrives later by the provider webhook (EP19); the rider polls EP16 or EP03 (payments).
Errors:

Status	Code	When	Database cause
409	trip_not_payable	the trip is not completed (and not cancelled with a fee)	trigger payment_enforce_rules, SQLSTATE RH022 (message "a fare payment needs a completed trip (status is %)", E13)
409	nothing_to_pay	the trip is cancelled with no fee	trigger SQLSTATE RH024 (message "a cancellation fee payment needs a cancelled trip with a fee above 0", E14)
409	payment_already_exists	a pending or succeeded payment already exists for this trip and purpose (made with another key)	unique index payment_one_live_per_purpose (E12)
502	provider_error	the provider refused or did not answer; or it returned a reference already used by another payment	none for the first two; the second is unique payment_provider_reference_key
400	invalid_idempotency_key	key missing or malformed	backstop idem_key_length (E32)
422	idempotency_key_reuse	same key, different body or different trip_id	none
409	request_in_progress	the same key is still running	key row without a stored response, or 55P03 (concurrency test)
422	validation_failed	payment_method_token missing or too long, unknown field	none

A wrong amount reaching the database (SQLSTATE RH023, "payment amount % does not equal the trip fare %", E15) is internal_error, because the API computes the amount. Also common.
Idempotency: keyed (1.7a), action pay_trip. A retry creates no second charge: the key row, the provider's idempotency key, and payment_one_live_per_purpose each refuse it.
Session settings: rider, sub.
File references: EP01 POST /v1/fare quotes is in docs/06_api_contracts.md at line 289. EP12 POST /v1/trips/{trip_id}/complete is in the same file at line 458. EP27 GET /v1/trips/{trip_id}/events is in the same file at line 665.

Appendix F, entity field tables

This appendix holds the field tables for 14 of the 16 entities, copied without change from 02_entities.md so a reader can see every column in one place. The two entities left out are trip (section 4 of 02_entities.md) and payment (section 8 of 02_entities.md); they are the two largest tables and are kept in 02_entities.md itself. The source line numbers below are from 02_entities.md as it stands at this fix.

rider (source: 02_entities.md lines 21 to 34)

A person who requests and pays for trips. Identifier: id. Serves U1, A1, A4, A5, B1, B8.

Field	Type	Req	Notes
id	uuid	required	primary key
full_name	text	required	personal data (B8); replaced on erasure
phone	text	required	personal data (B8); E.164 format such as +2348012345678; replaced on erasure
email	text	optional	personal data (B8); cleared on erasure

rating_sum	integer	required	SPEED cache, default 0: total stars received from drivers
rating_count	integer	required	SPEED cache, default 0: number of ratings received
deleted_at	timestamptz	optional	set when the account is erased (soft delete)
created_at	timestamptz	required	
updated_at	timestamptz	required	

driver (source: 02_entities.md lines 36 to 49)

A person who accepts and drives trips. Identifier: id. Serves U2, A2, A3, A5, B2, B8.

Field	Type	Req	Notes
id	uuid	required	primary key
full_name	text	required	personal data (B8)
phone	text	required	personal data (B8)
licence_number	text	required	personal data (B8); needed to vet a driver
rating_sum	integer	required	SPEED cache, default 0
rating_count	integer	required	SPEED cache, default 0
deleted_at	timestamptz	optional	soft delete
created_at	timestamptz	required	
updated_at	timestamptz	required	

vehicle (source: 02_entities.md lines 51 to 66)

A car that belongs to one driver. Identifier: id. Serves U2, A2, B2, B8.

Field	Type	Req	Notes
id	uuid	required	primary key
driver_id	uuid	required	foreign key to driver
plate_number	text	required	the number plate; unique while not deleted
make	text	required	
model	text	required	
colour	text	optional	
seat_count	smallint	required	1 to 8
is_active	boolean	required	default true; false means the driver no longer uses it
deleted_at	timestamptz	optional	soft delete
created_at	timestamptz	required	
updated_at	timestamptz	required	

trip_status_history (source: 02_entities.md lines 95 to 112)

One row for every status change of a trip, written by the database itself (B5). Append only. Identifier: id. Serves B5, A1, A2, A3.

Field	Type	Req	Notes
id	uuid	required	primary key
trip_id	uuid	required	foreign key to trip
from_status	text	optional	empty on the first row (creation)
to_status	text	required	
actor_role	text	required	rider, driver, support or system
actor_rider_id	uuid	optional	foreign key to rider; set when actor_role is rider
actor_driver_id	uuid	optional	foreign key to driver; set when actor_role is driver
actor_admin_id	uuid	optional	foreign key to support_admin; set when actor_role is support
created_at	timestamptz	required	the time of the change
updated_at	timestamptz	required	equals created_at (append only)
seq	bigint	required	GENERATED ALWAYS AS IDENTITY (migration 010). The database numbers each new row in insert order, and nobody can supply a value. Timelines are ordered by seq, not by created_at: every move made inside one transaction has the same created_at

location_ping (source: 02_entities.md lines 118 to 130)

One GPS point sent by the driver's phone while a trip is in progress. Append only except for scheduled deletion. Identifier: id. Serves A3, B8.

Field	Type	Req	Notes
id	uuid	required	primary key
trip_id	uuid	required	foreign key to trip; the driver is found through the trip
latitude	numeric(8,6)	required	-90 to 90
longitude	numeric(9,6)	required	-180 to 180
accuracy_m	integer	optional	phone's stated accuracy in metres; not negative
recorded_at	timestamptz	required	time on the phone
created_at	timestamptz	required	time the server stored it
updated_at	timestamptz	required	equals created_at

fare_quote (source: 02_entities.md lines 132 to 149)

The price estimate shown to the rider before the trip exists. It is never edited. Identifier: id. Serves A1, B3.

Field	Type	Req	Notes
id	uuid	required	primary key
rider_id	uuid	required	foreign key to rider
pickup_latitude	numeric(8,6)	required	
pickup_longitude	numeric(9,6)	required	
dropoff_latitude	numeric(8,6)	required	
dropoff_longitude	numeric(9,6)	required	
estimated_distance_m	integer	required	not negative
estimated_duration_s	integer	required	not negative
amount_minor	bigint	required	estimated fare in kobo
currency	char(3)	required	NGN
expires_at	timestamptz	required	after this the quote cannot start a trip
created_at	timestamptz	required	
updated_at	timestamptz	required	equals created_at

refund (source: 02_entities.md lines 168 to 181)

Money returned to the rider against one payment, whole or part. Never edited. Identifier: id. Serves B3, B4, B10, U3.

Field	Type	Req	Notes
id	uuid	required	primary key
payment_id	uuid	required	foreign key to payment
amount_minor	bigint	required	kobo; more than 0
currency	char(3)	required	NGN
reason	text	required	why support refunded
created_by_admin_id	uuid	required	foreign key to support_admin
provider_reference	text	required	the provider's refund id; unique. A refund row is created only after the provider confirms it (assumption 5)
created_at	timestamptz	required	
updated_at	timestamptz	required	equals created_at

rating (source: 02_entities.md lines 183 to 194)

One person's rating of the other after a completed trip. Never edited. Identifier: id. Serves A5, B7.

Field	Type	Req	Notes
id	uuid	required	primary key
trip_id	uuid	required	foreign key to trip
rater_role	text	required	rider or driver. The rater and the person rated are both found through the trip, so nothing is copied
stars	smallint	required	1 to 5
comment	text	optional	up to 500 characters; personal data (B8)
created_at	timestamptz	required	
updated_at	timestamptz	required	equals created_at

idempotency_key (source: 02_entities.md lines 196 to 212)

A client chosen key that makes a retry safe. Stores the first answer so a retry gets it back unchanged. Identifier: id. Serves A1, A4, B6.

Field	Type	Req	Notes
id	uuid	required	primary key
rider_id	uuid	required	foreign key to rider
action	text	required	request_trip or pay_trip
key	text	required	the client's key, 1 to 128 characters
request_hash	text	required	SHA-256 of the request body, 64 hex characters; a retry with the same key but a different body is refused
response_status	smallint	optional	HTTP status of the first answer; empty while the first request is still running
response_body	jsonb	optional	the first answer, returned unchanged on a retry
trip_id	uuid	optional	foreign key to trip, set when the action made a trip
payment_id	uuid	optional	foreign key to payment, set when the action made a payment
expires_at	timestamptz	required	the row is deleted after this (04, question 4)
created_at	timestamptz	required	
updated_at	timestamptz	required	changes once, when the response is stored

setting (source: 02_entities.md lines 214 to 225)

Named whole number settings that the owner can change without a code release. Identifier: key (text), not a uuid. Reason: it is a short fixed name the code looks up (for example cancellation_fee_minor). It is never created by users and never shown in a URL, so a random id adds nothing. Serves B9, B7, A1, B3.

Field	Type	Req	Notes
key	text	required	primary key
int_value	bigint	required	not negative. Holds two kinds of value: kobo for keys ending in _minor, and degrees multiplied by 1,000,000 for the four service_area_*_e6 keys. Other keys hold plain counts (days, seconds). The key name says which
currency	char(3)	optional	NGN; required exactly when the key ends in _minor
description	text	required	plain English meaning
updated_by_admin_id	uuid	optional	foreign key to support_admin
created_at	timestamptz	required	
updated_at	timestamptz	required	

support_admin (source: 02_entities.md lines 229 to 240)

A staff member who investigates trips, cancels and refunds. Identifier: id. Serves U3, B9, B10.

Field	Type	Req	Notes
id	uuid	required	primary key
full_name	text	required	personal data (B8)
email	text	required	unique; personal data (B8)
is_active	boolean	required	default true
deleted_at	timestamptz	optional	soft delete
created_at	timestamptz	required	
updated_at	timestamptz	required	

audit_log (source: 02_entities.md lines 242 to 258)

A permanent record of sensitive actions: setting changes, payment status changes, refunds, cancellations by support, and support notes about disputes. Append only. Identifier: id. Serves B5, B4, U3, and the dispute rule in section 4 of the requirements.

Field	Type	Req	Notes
id	uuid	required	primary key
actor_role	text	required	rider, driver, support, provider or system
actor_rider_id	uuid	optional	foreign key to rider

actor_driver_id	uuid	optional	foreign key to driver
actor_admin_id	uuid	optional	foreign key to support_admin
action	text	required	for example status_change, setting_change, refund_created, dispute_note
entity_type	text	required	trip, payment, refund, rating, rider, driver, setting or provider_event. provider_event was added in Stage 3: a provider message about a reference we do not know has no payment or refund to cite, so the audit row cites the event itself
entity_id	uuid	optional	the id of the thing acted on (not a foreign key, because it points to different tables). Required for every entity_type except setting, whose key is text and is stored in details
note	text	optional	a support note. Written only through the function add_support_note (see below). The rule that it must not contain card data or secrets is application validation only. The database does not check it
details	jsonb	optional	for example {"from":"pending","to":"succeeded"}
created_at	timestamptz	required	
updated_at	timestamptz	required	equals created_at

status_transition_rule (source: 02_entities.md lines 264 to 274)

One row per legal status change for a given actor, for the trip and for the payment. Identifier: the whole row (entity, from_status, to_status, actor_role) is the primary key, because the row has no existence apart from those four values. Serves B5, B9, A2, A3, A4.

Field	Type	Req	Notes
entity	text	required	trip or payment
from_status	text	required	
to_status	text	required	
actor_role	text	required	rider, driver, support, provider or system
created_at	timestamptz	required	
updated_at	timestamptz	required	

provider_event (source: 02_entities.md lines 278 to 294)

One row for each message the payment provider sends us (a payment or refund result). It is how a repeated or contradicting message is recognised (U4, A4, B6). Identifier: id (our uuid). The provider's own id, provider_event_id, is unique. Serves U4, A4, B5, B6.

Field	Type	Req	Notes
id	uuid	required	primary key
provider_event_id	text	required	the provider's own id for this message; unique. A repeated message carries the same value
provider_reference	text	required	the provider's id of the payment or refund the message is about. Matches payment.provider_reference or refund.provider_reference by value. It is not a foreign key, because a message about an unknown reference must still be stored
event_type	text	required	payment_succeeded, payment_failed, payment_not_found (the provider answered a reconciliation check with "no record of this payment") or refund_succeeded
amount_minor	bigint	optional	kobo, more than 0; filled together with currency

currency	char(3)	optional	NGN; filled together with the amount
occurred_at	timestamptz	optional	the provider's own time of the event, as the provider states it. Not trusted for ordering, only kept for evidence
received_at	timestamptz	required	when we received the message. Default clock_timestamp(), not now(). Reason: now() is the start time of the transaction, so every event stored in one transaction would get the same value, and the rule P3 ("not older than the latest provider event of any type for that reference") could not tell which event came last. clock_timestamp() gives each row its own moment
processed_at	timestamptz	optional	when we finished handling it; filled together with outcome
outcome	text	optional	applied, ignored_duplicate or needs_support; filled together with processed_at
created_at	timestamptz	required	
updated_at	timestamptz	required	changes once, when processed_at and outcome are filled